



Software Engineering

5430

Spring 2023



by:

Abdelnasser Ouda

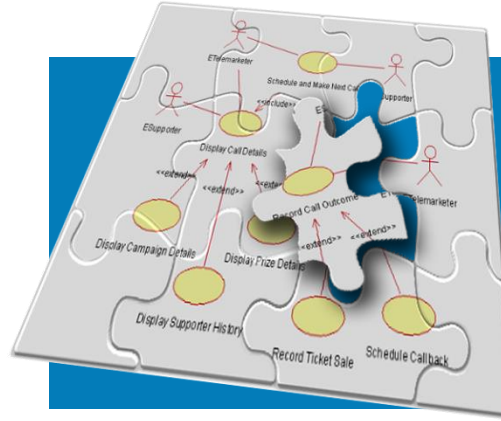
Computer Science Department

Email: abdelnasser.ouda@unt.edu

Discovery Park F203

Unit 1: Introduction to Software Engineering

- Software Engineering Definition & Introduction
- Introduction to Software Process
- Software Development LifeCycle (SDLC)
 - Activities
 - Modeling
 - Methodology
- Agile software development methodology
- Selecting the Appropriate Development Methodology



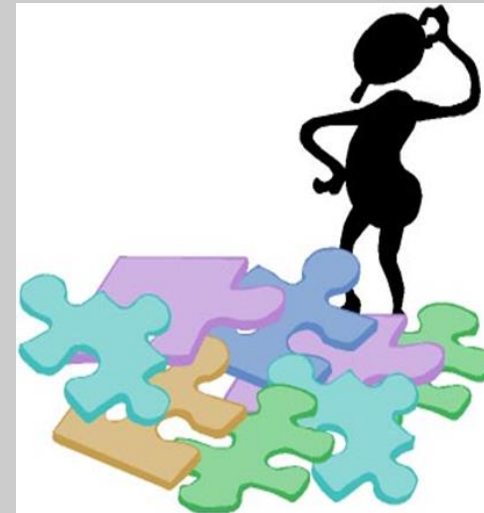
Software Engineering Definition & Introduction

Software Engineering Definition

- Software Engineering is more than writing program code.
- So, **what is Software Engineering?**

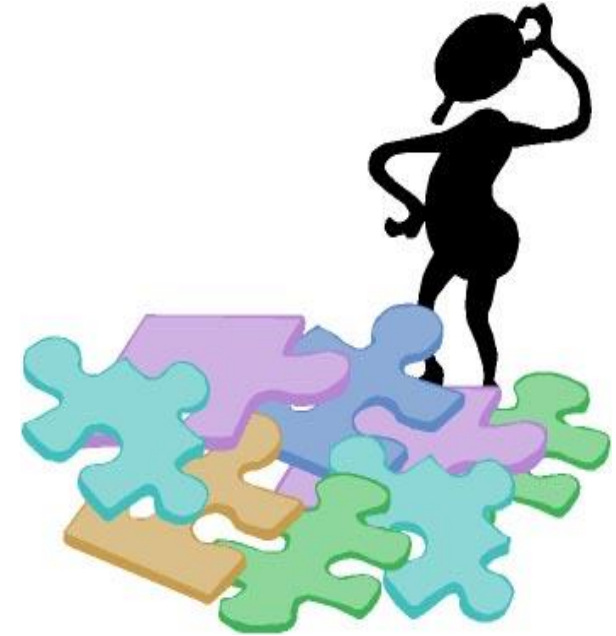
Software Engineering / Development is

➤ Problem solving



Software Engineering Defined (1)

- Software engineering is a **problem-solving** activity.
- Includes five steps:
 1. formulate the problem
 2. analyze the problem
 3. search for solutions
 4. decide on the appropriate solution
 5. justify the solution

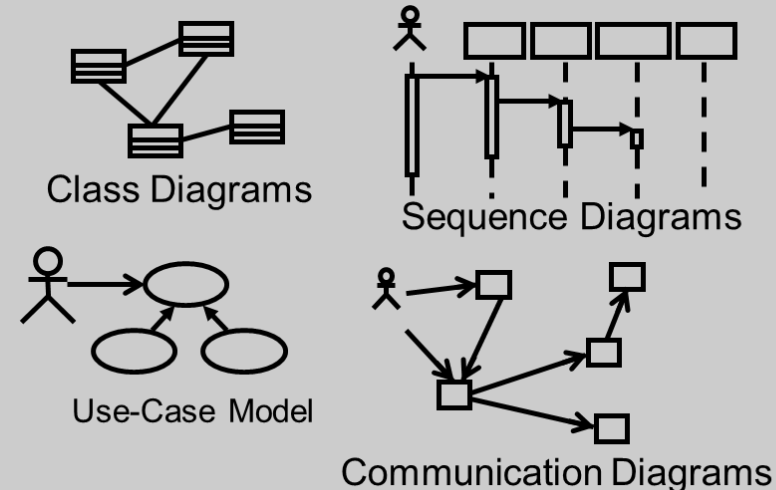


Software Engineering Definition

- Software Engineering is more than writing program code.
- So, **what is Software Engineering?**

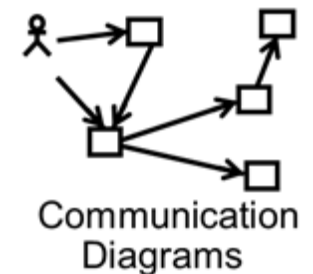
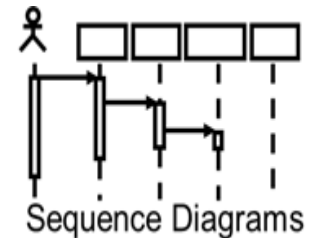
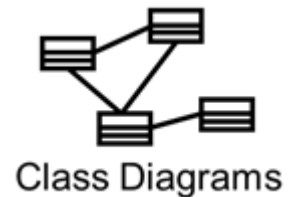
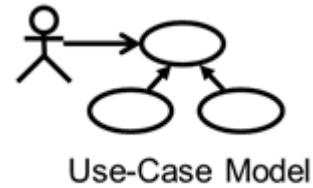
Software Engineering / Development is

- Problem solving
- Modeling



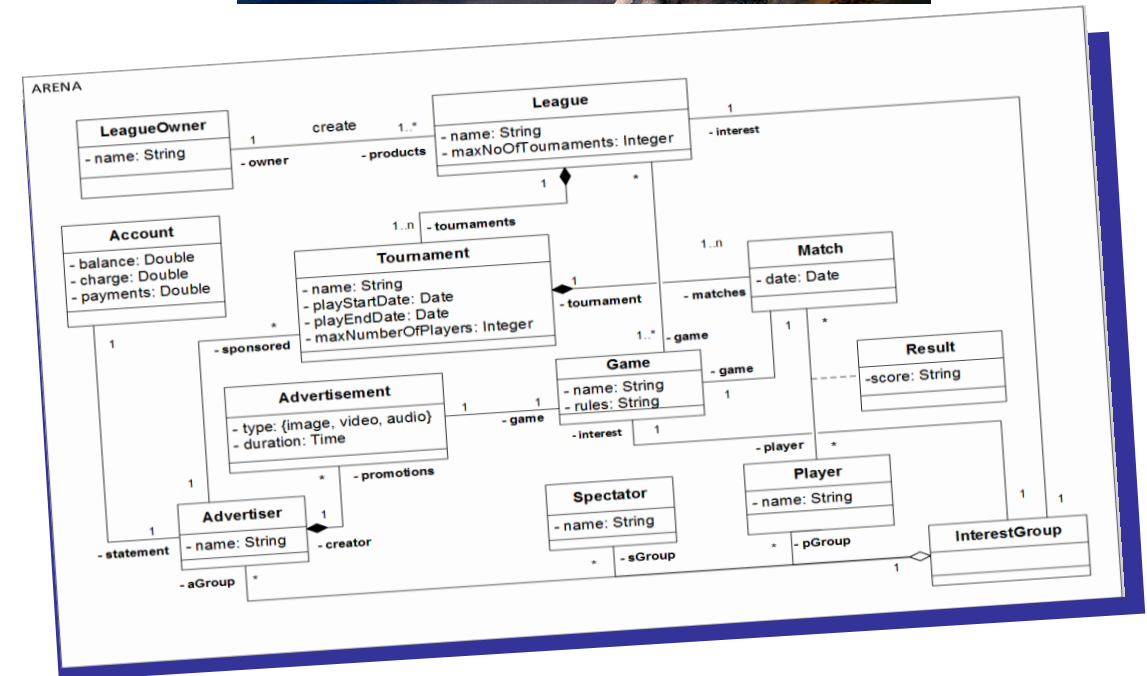
Software Engineering Defined (2)

- Software engineering is a **modeling** activity
- **Modeling** is an important problem-solving technique, especially in software design.
- Software engineers deal with complexity through modeling,
 - *by focusing at any one time on only the relevant details and ignoring everything else.*
- A model is an **abstract** representation of a system that enables us to answer questions about the system.
- Models are useful when dealing with systems that are too large, too complicated, or too expensive to experience firsthand.



Abstraction and Modeling

- A model also allow us to visualize and understand systems that:
 - No longer exists
 - An existing system
 - A future system to be built.



Software Engineering Definition

- Software Engineering is more than writing program code.
- So, **what is Software Engineering?**

Software Engineering / Development is

- Problem solving
- Modeling
- Knowledge acquisition



Software Engineering Defined (3)

- Software engineering is a **knowledge acquisition** activity.
- In modeling the application and solution domain, you as a software engineer
 - collect data,
 - organize it into information,
 - formalize it into knowledge,
 - and build strategic decisions from it.
- Knowledge acquisition **is nonlinear**, as a single piece of data can invalidate complete models.



Software Engineering Definition

- Software Engineering is more than writing program code.
- So, **what is Software Engineering?**

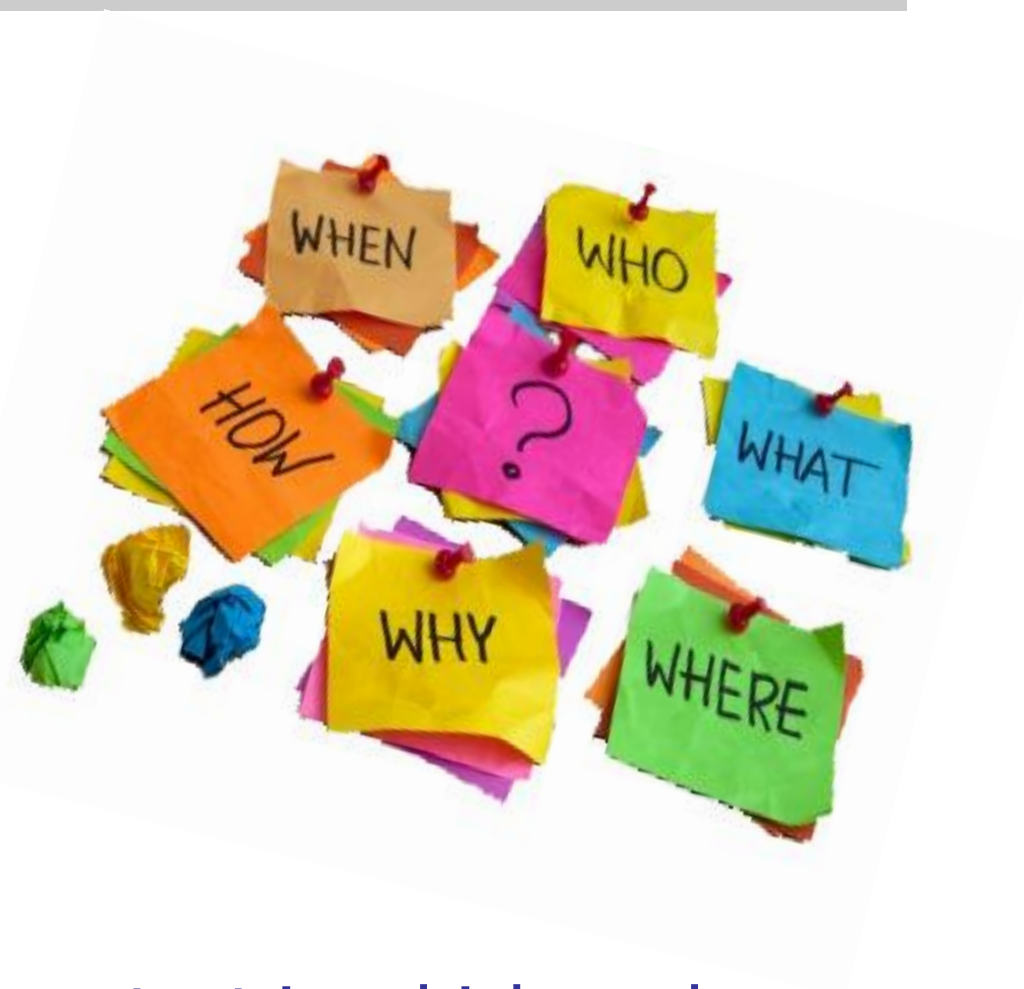
Software Engineering / Development is

- Problem solving
- Modeling
- Knowledge acquisition
- Rationale management



Software Engineering Defined (4)

- Software engineering is a **rational-driven** activity.
- When a decision about the system solution is made,
 - the rational behind this decision should be captured and described.
- To change the system, it is not enough to understand its current components and behavior.
 - It is also necessary to understand the context in which each design decision was made.



Software Engineering Definition

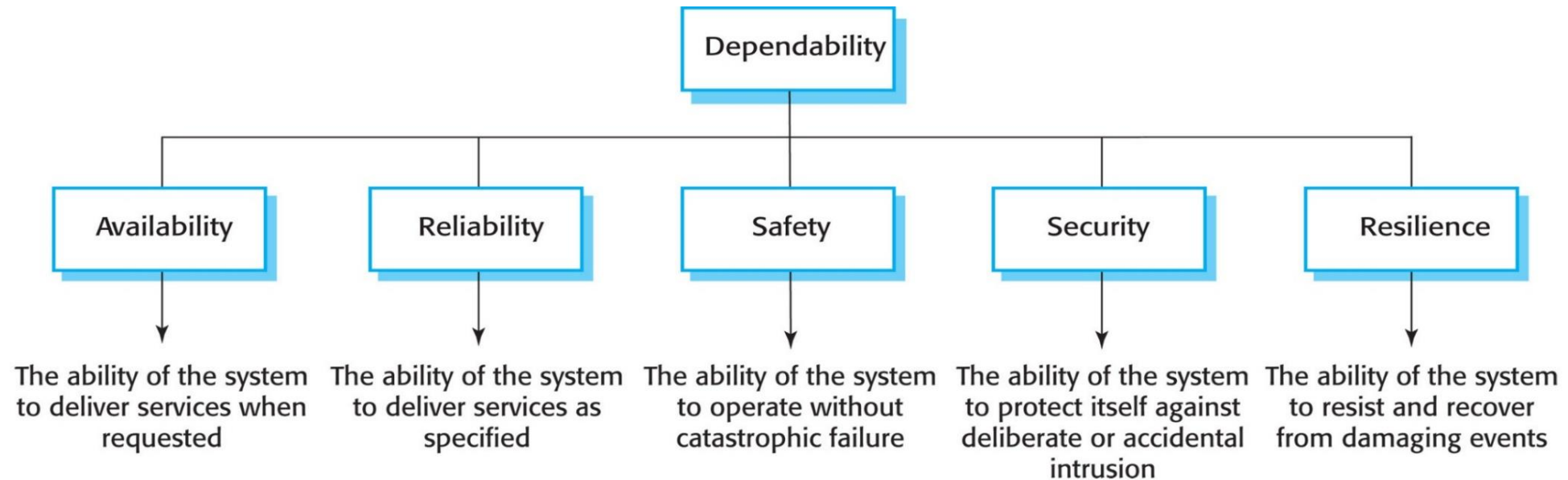
- Software Engineering is more than writing program code.
- So, **what is Software Engineering?**

Software Engineering / Development is

- Problem solving
- Modeling
- Knowledge acquisition
- Rationale management
- Quality Control



Software Engineering Defined (5)



Where are we?

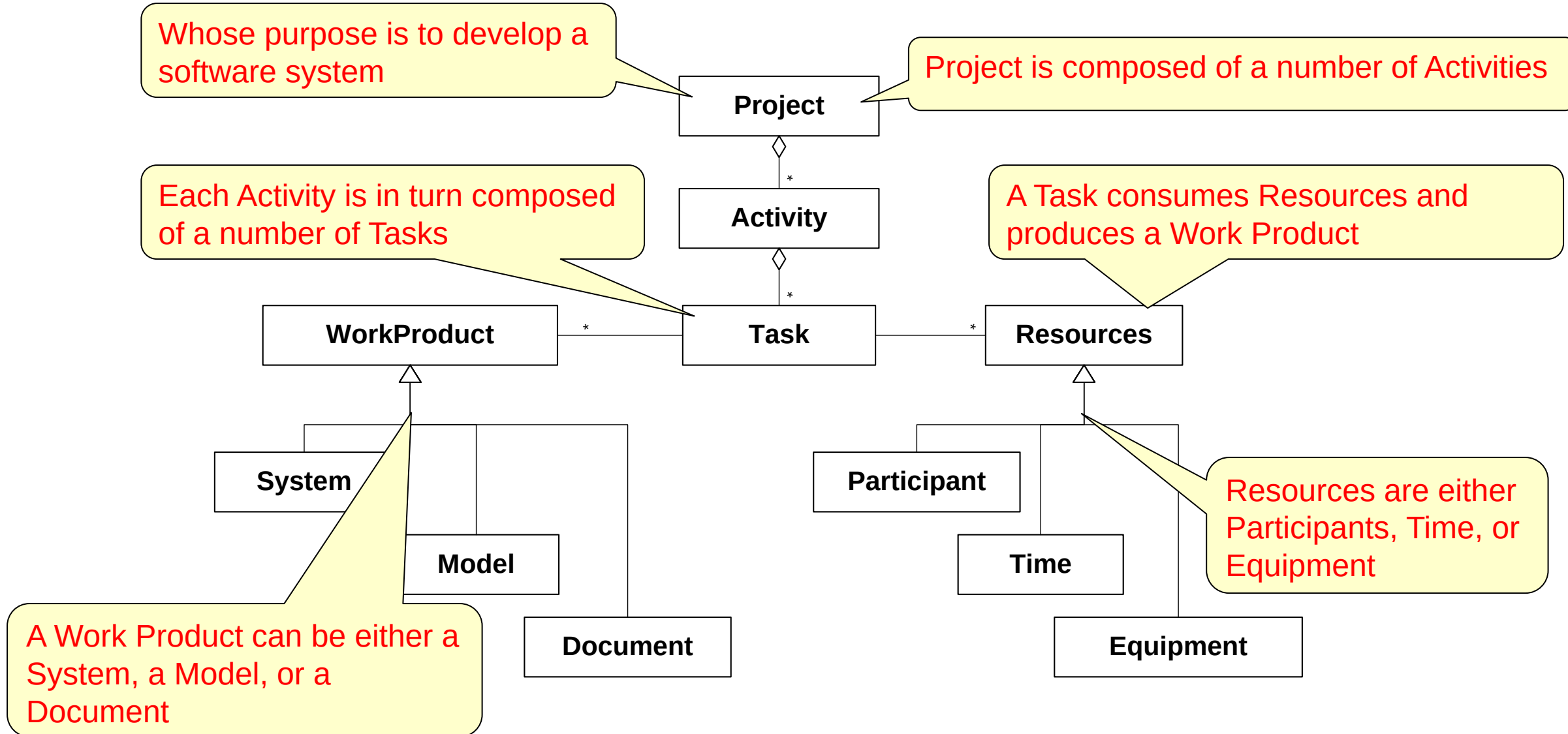
- We just defined a Software Engineering as:

Software Engineering / Development is

- Problem solving
- Modeling
- Knowledge acquisition
- Rationale management
- Quality Control

- We understand the important of software engineering
- Did I mention? **Software Engineering is more than writing program code.**
- **Let us now describe the main terms and concepts of software Engineering.**

Software Engineering Concepts





Introduction to Software Process

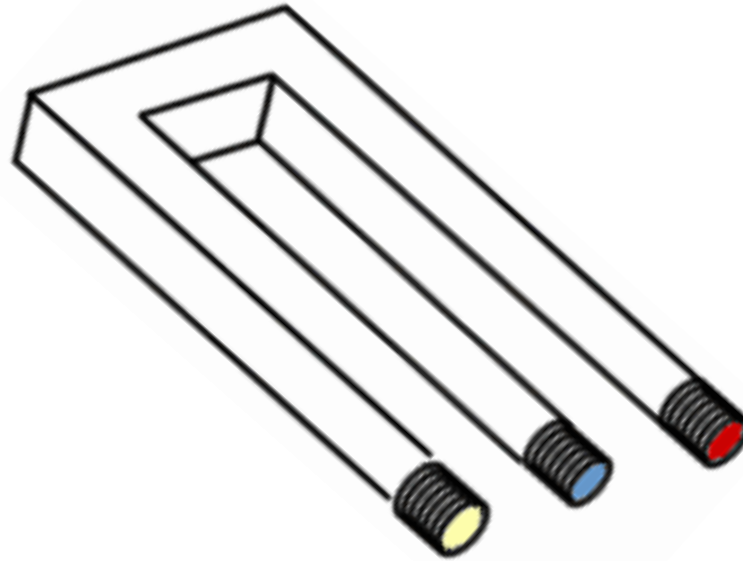
The development challenges

- The goal of software engineering is to develop software systems (obvious).

But

- Some systems are impossible to develop,
 - because the requirements are not clear or the technology is not available.
 - For example, consider the following requirements.

Can you develop this system?

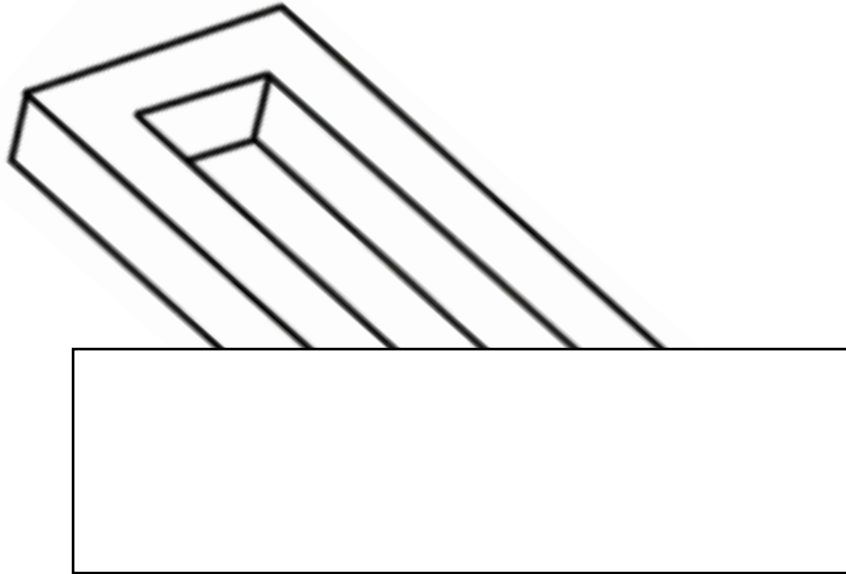


- First immediate answer is NO, because it is physically impossible
- But we are developers, **let's try**.

If we can build it, and if not, what the problem is.

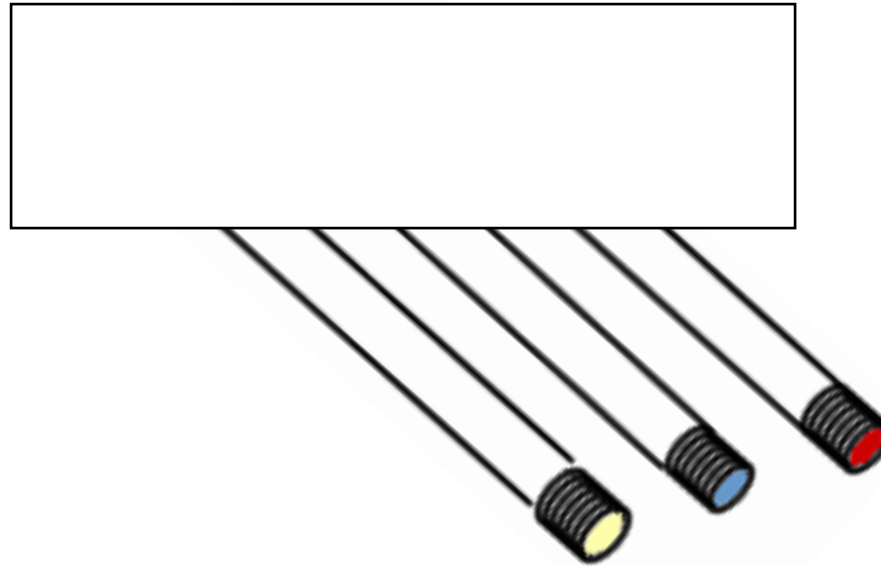
Maybe we can fix the original problem specification

Can you develop this system?



- When you cover the bottom half, we will find that the top part of the figure is entirely possible.
- In this case, we interpret the foreground figure as being built of **flat surfaces** forming **two** rectangular prongs.

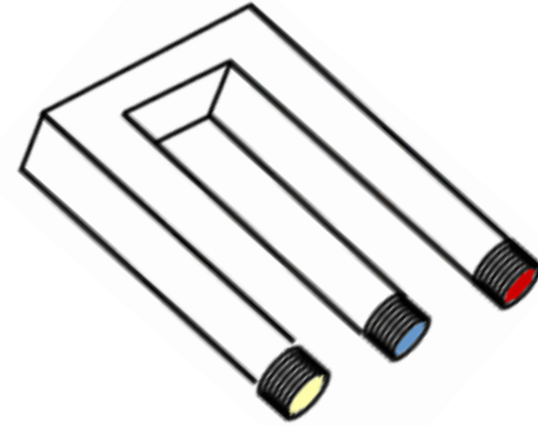
Can you develop this system?



- When we cover the upper half, we will find that the bottom part of the figure is also possible.
- In this case, we interpret this figure as built of **curved surfaces** forming **three** separate cylindrical lines.

Can you develop this system?

- Software analyst says:
 - The two parts that are joined in this figure have different interpretations of their shapes.
 - So the answer is: **No**, we cannot build this as it is drawn
 - It has **ambiguous** surfaces.
 - In software engineering terminology the drawing has **ambiguous requirements**.



The Software Analyst

- The software analyst assists and **guides** the software project team so the team develops the **right system** in an effective way.
- Software analysts must **understand how to apply technology** in order to solve problems.
- Systems analysts may also serve as **change agents** who identify the organization improvements needed, design systems to implement those changes, and train/motivate others to use the systems.

What is Software Design?

- Software designers do what designers in other disciplines do, except they do it for software systems.
- So, what is Software Design?

Software design is the activity of specifying the nature and composition of software systems that satisfy client needs and desires, subject to constraints.

What is Software System?

A **Software System** is an entity comprised of one or more programs, data, supporting materials and services that satisfies client needs and desires.

The Development Challenges

- The goal of software engineering is to develop software systems (obvious).

But

- Some systems are impossible to develop,
 - because the requirements are not clear or the technology is not available.
- Sometimes the systems are difficult to develop,
 - Why?

Why is Software Development Difficult?

- The problem domain (also called application domain) is difficult,
 - just because we are not experts in it.
- Sometimes solution domain is difficult
 - because it using experimental technology, that has not matured yet.
- Requirements are constantly changing
 - The client might not know all the requirements in advance
 - Frequent changes are difficult to manage
- There is more than one software system
 - New system must often be backward compatible with existing system ("legacy system")

The Development Challenges

- The goal of software engineering is to develop software systems (obvious).

But

- Some systems are impossible to develop,
 - because the requirements are not clear, or the technology is not available.
- Sometimes the systems are difficult to develop.

Let's view these problems as the nonfunctional requirements for a system that supports software development (e.g., CASE Tools)

This leads us to software life cycle modeling

Functional & non-Functional Requirements

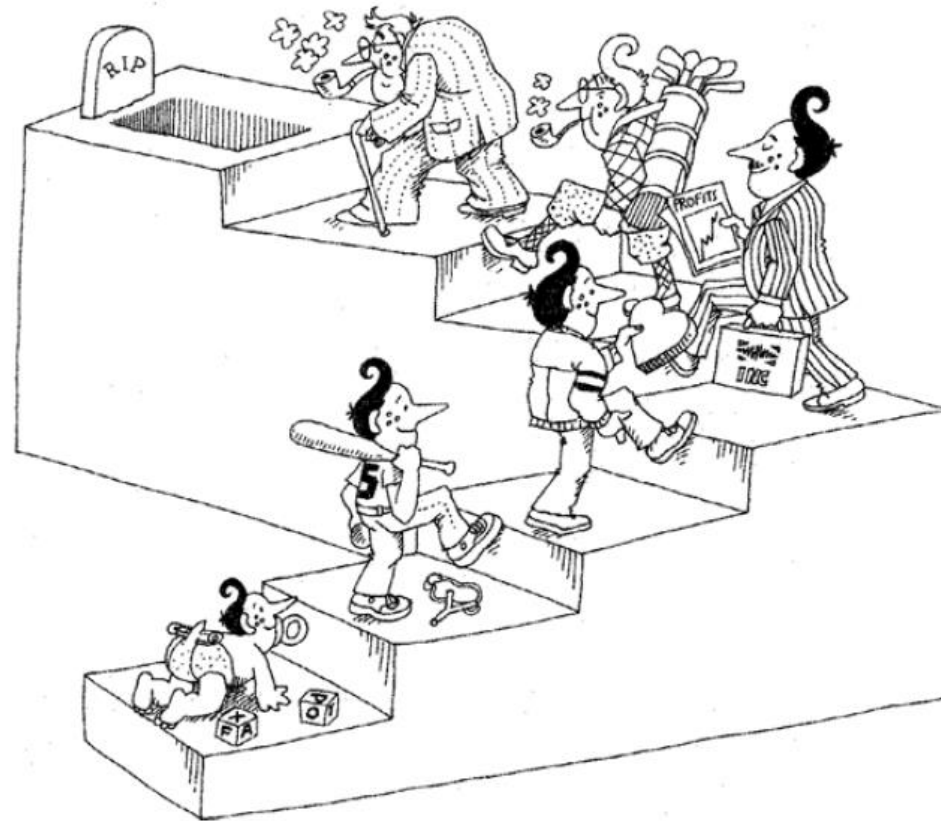
- A **functional requirement** is an area of functionality that the system must support.
 - For example, the student registration system: “The system shall allow a student to add, or drop a course online” is a functional requirement.
- A **nonfunctional requirement** is a constraint on the operation of the system.
 - Whereas “The system should provide feedback to the student in less than one second” is a nonfunctional requirement



Software Development LifeCycle (SDLC)

Software Life Cycle

- The term “Lifecycle” is based on the metaphor of the life of a person:



BOOCH, G. (1994): *Object Oriented Analysis and Design with Applications*, 2nd ed,

Software Development LifeCycle (SDLC)

- **Software development lifecycle** identifies:
 - the exact **activities** of developments from initial inception to phasing it out
 - the applied **modeling approach**
 - the **method** (methodology) and associated development process

SDLC - Activities

- **Questions to ask:**

- What is the problem?
- Is the problem fully described?
- What is the solution?
- What are the best mechanisms to implement the solution?
- How is the solution constructed?
- Is the problem solved?
- Can the customer use the solution?
- How do we deal with changes that occur during the development? Are enhancements needed?

Identifying Software Development Activities

Requirements Elicitation

Analysis

System Design

- What is the problem?
- Is the problem fully described?
- What is the solution?

Application
Domain

Object Design

Program Implementation

Testing

Delivery

Maintenance

- What are the best mechanisms to implement the solution?
- How is the solution constructed?
- Is the problem solved?
- Can the customer use the solution?
- Are enhancements needed?

Solution
Domain



SDLC - Requirements Elicitation Activity

- During requirements elicitation, the client and developers **define the purpose of the system.**
- The result of this activity is a description of the system in terms of **actors and use cases.**
 - **Actors** represent the external entities that interact with the system
 - Actors include roles such as end users, other computers the system needs to deal with.
 - **Use cases** are general sequences of events that describe all the possible actions between an actor and the system for a given piece of functionality.

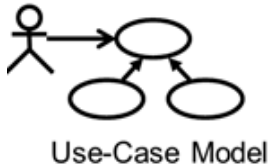
Tickets distribution system

Consider a machine that distributes tickets for a train. Travelers have the option of selecting a ticket for a single trip, for multiple trips, or a time card for a day or a week. The ticket distributor computes the price of the requested ticket based on the area in which the trip will take place and whether the traveler is a child or an adult.

In this example, travelers purchasing tickets are **end users**. The train company, contracting the development of the ticket distributor, is **the client** of the project. The engineers realizing the system (and the software), John, Marc, and Zoe, are **developers**. Their boss, Alice, coordinating the work and the communication with the client, is the **project manager**.

Next slide depicts a use case for the ticket distributor example.

An example of use case **PurchaseOneWayTicket**

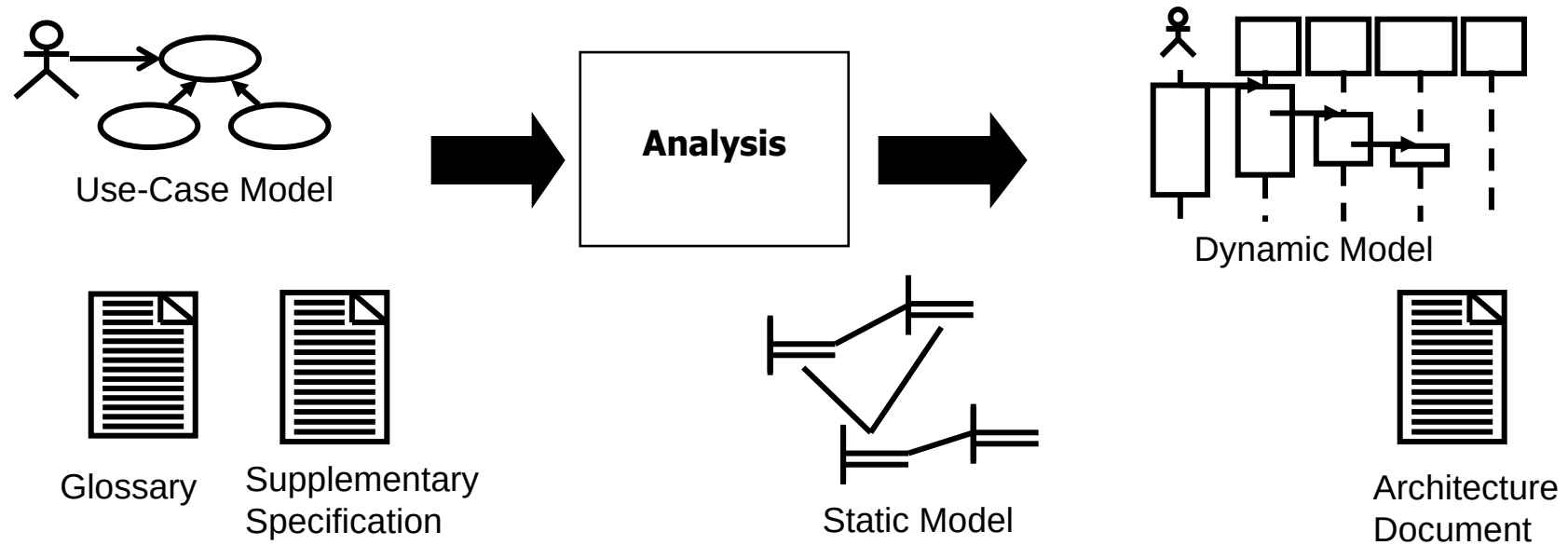


We will describe requirements elicitation, including use cases and nonfunctional requirements, in details during in Unit 2.

Use case name	PurchaseOneWayTicket
Participating actor	Initiated by Traveler
Entry condition	• The Traveler stands in front of the ticket distributor, which may be located at the station of origin or at another station.
Flow of events	1.The Traveler selects the source and destination stations. 2.The TicketDistributor displays the price of the ticket. 3.The Traveler inserts an amount of money that is at least as much as the price of the ticket. 4.The TicketDistributor issues the specified ticket to the Traveler and returns any excess change.
Exit condition	• The Traveler holds a valid ticket and any excess change.
Quality requirements	• If the transaction is not completed after 1 minute of inactivity, the TicketDistributor returns all inserted change.

SDLC - Analysis Activity

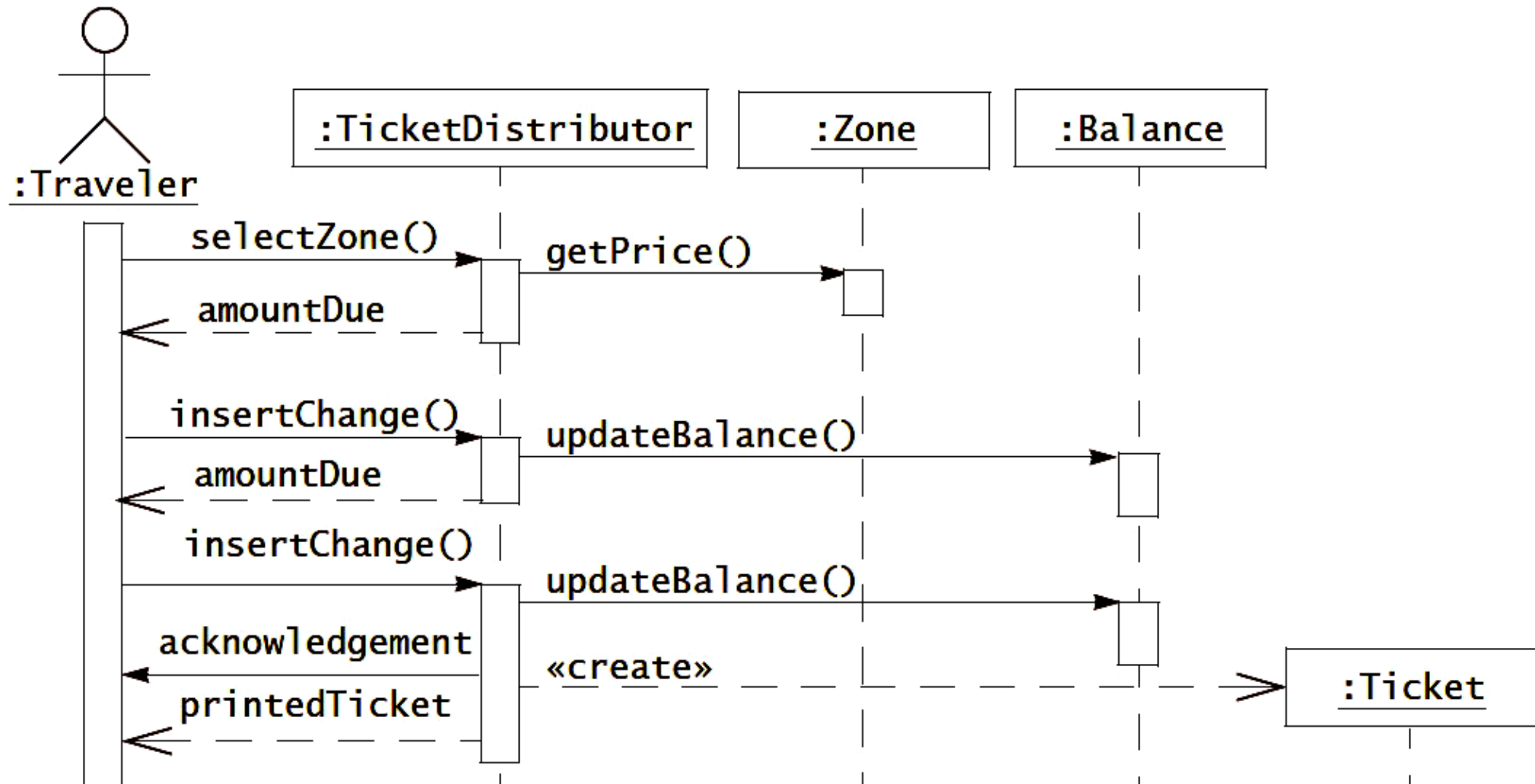
- During **analysis**, developers aim to produce a model of the system that is correct, complete, consistent, unambiguous, realistic, and verifiable.



- Developers transform the use cases produced during requirements elicitation into an object model that completely describes the system

Next slide depicts an example of dynamic model for the ticket distributor .

An example of dynamic model for the TicketDistributor



SDLC - Analysis Activity ...

Analysis model describes **user requirements** by the following products:

- **a use case model**, describing the system functionality from actors' point of view
- **an object model**, describing the entities manipulated by the system. E.g, the Class diagrams: Describe the static structure of the system: Objects, attributes, associations.
- **a sequence diagram** for each use case, showing the sequence of interactions among objects participating in the use case. Describe the dynamic behavior between objects of the system
- **a statechart diagram** for each use case, showing the sequence of interactions. Describe the dynamic behavior of an individual object

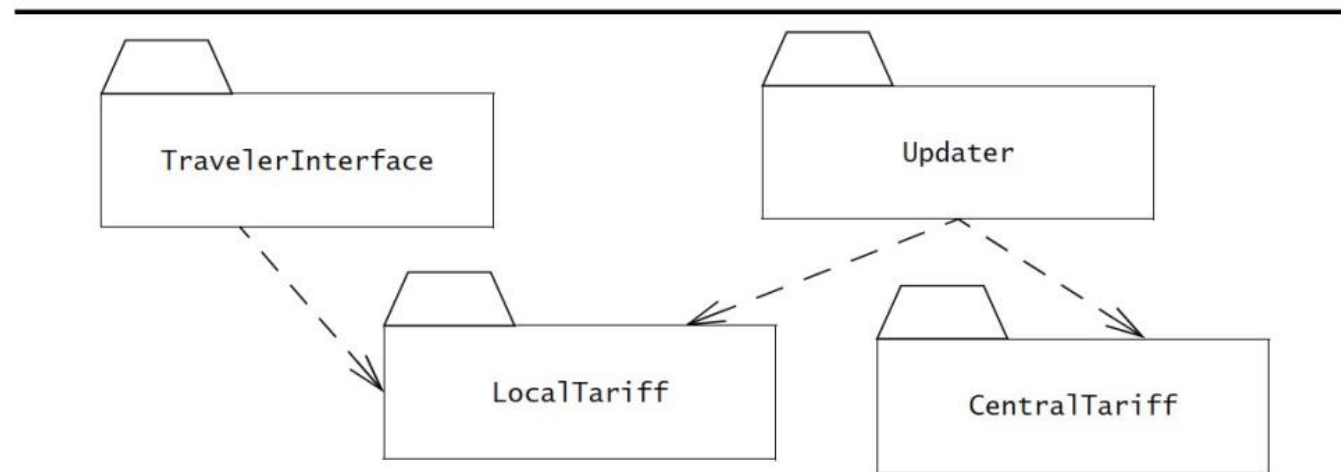
SDLC - Analysis Activity ...

- The analysis model, however, does not contain information about the internal structure of the system, its hardware configuration, how the system should be realized.

System design is the first step in this direction.

SDLC - **System Design** Activity

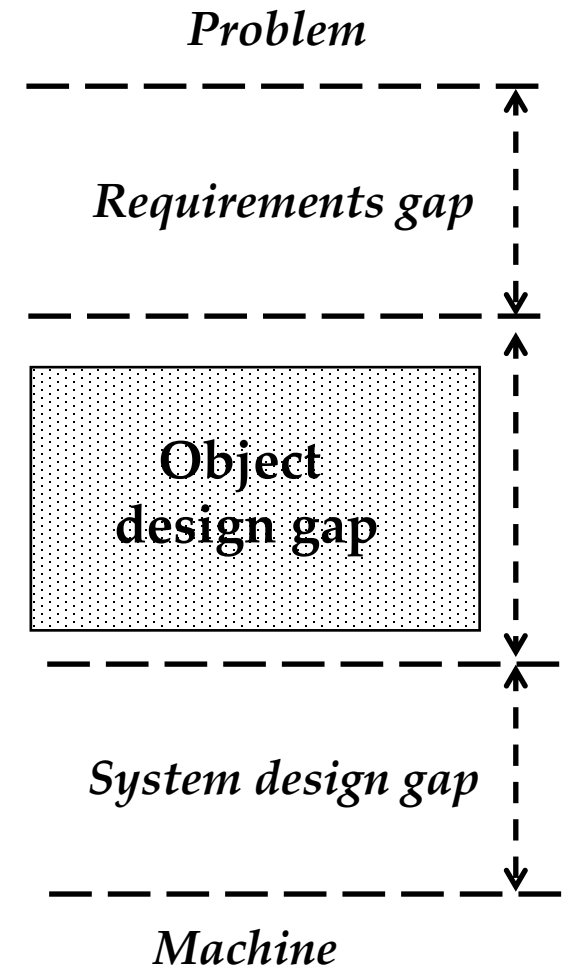
- During **system design**, developers
 - define the design goals of the project and
 - decompose the system into smaller subsystems that can be realized by individual teams.
- Elements such as, hardware/software platform, database engine, access control policy will also be defined during system design.



An example of system decomposition for the ticket distributor.

SDLC - Object Design Activity

- During **object design**, developers define custom objects to bridge the gap between the analysis model and the hardware/software platform defined during system design.
- The result of the object design activity is a detailed object model providing precise descriptions for each element.



We will describe object design and its related concepts, in detail in Unit 4.

SDLC - **Implementation** Activity

- During implementation, developers translate the solution domain model into source code.
- This includes implementing the attributes and methods of each object and integrating all the objects such that they function as a single system.
- Software Architecture used:
 - **Layered style.**
 - **Model-View-Controller (MVC) style.**
 - **Pipe-and-Filter.**
 - **Repository style.**
 - **Hybrids of styles.**

We will discuss the mapping of UML models to code, in detail in Unit 5.

SDLC - Testing Activity

- During testing, developers find differences between the system and its models by executing the system (or parts of it) with sample input data sets.
- The goal of testing is to discover as many faults as possible such that they can be repaired before the delivery of the system.
- The planning of test phases occurs in parallel to the other development activities:
 - System tests are planned during requirements elicitation and analysis,
 - integration tests are planned during system design, and
 - unit tests are planned during object design.

We will discuss the Testing activities, in detail in Unit 6.

SDLC - Maintenance Activity

- Maintenance is performed for two reasons:
 - Removing software errors
 - Enhancing existing software
 - users request additional features
 - business changes over time
 - hardware and software change
- Over time the cost of continued maintenance will be greater than that of creating an entirely new system. At that point it becomes more feasible to perform a new systems study.

We will discuss the maintenance activity and evolution in detail in Unit 7.

Software Development LifeCycle (SDLC)

- **Software development lifecycle** identifies:
 - the exact **activities** of developments from initial inception to phasing it out
 - the applied **modeling approach**
 - the **method** (methodology) and associated development process

Modeling approach

- A model is an **abstract** representation of a system that enables us to answer questions about the system.
- Software engineers deal with complexity through modeling,
 - by focusing at any one time on only the relevant details and ignoring everything else.
- Conventional software has been well served by the so-called **structured approach**
- Modern interactive GUI systems require object programming and the **object approach** is the best way to design such systems

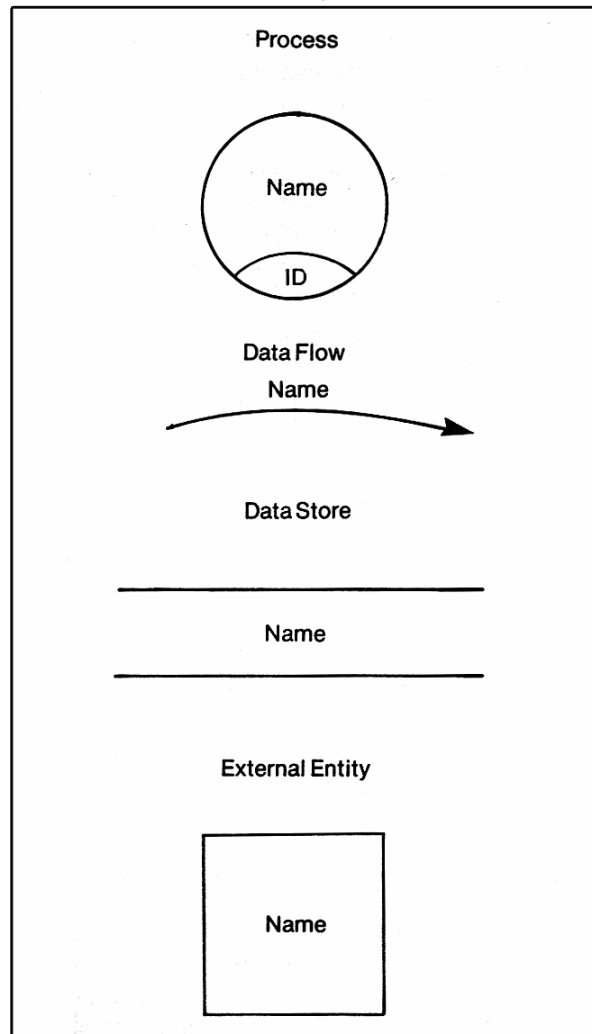
Structured Approach

- Popularized in 1980s
- Based on two techniques :
 - DFD (data flow diagrams) for process modeling
 - ERD (entity relationship diagrams) for data modeling
- Process-centric
 - Brakes system down to manageable units in the activity called functional decomposition
- From Process-centered to Data-centered
 - direct result of the popularity of the relational database model
- The combination of DFDs and ERDs delivers relatively complete analysis models that capture all of a system's functions and data at a desirable level of abstraction

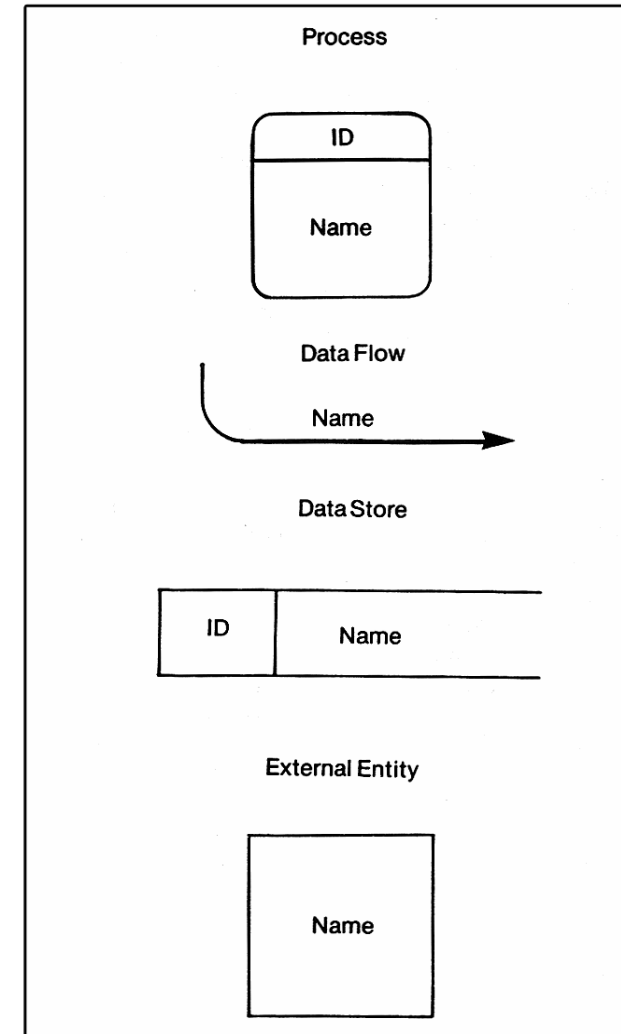
Data Flow Diagram

- A data flow diagram (DFD) is a visualization of the data interaction with other activities
 - Within the overall system or some lower-level part of it
- On a DFD a reader will find graphic representations of
 - external entities,
 - processes,
 - data flows, and
 - data stores
- interconnected to show the progressive transformation of data.

DFD Notations



Constructs of Yourdon-DeMarco



Constructs of Gane and Sarson

DFD – Data Process Interactions

- Once client and developer reach consensus on the scope of the target system,
- The developer can proceed to identify the data-process interaction in progressively finer detail.
 - Each resulting DFD, at what are called the "intermediate levels," is an elaboration or explosion of data-process interaction at the prior level.
- For instance, the single process in the context level DFD shown in Figure 1, is now exploded into a diagram, such as that shown in Figure 2 that visualizes its inner workings.

DFD – Data Process Interactions ...

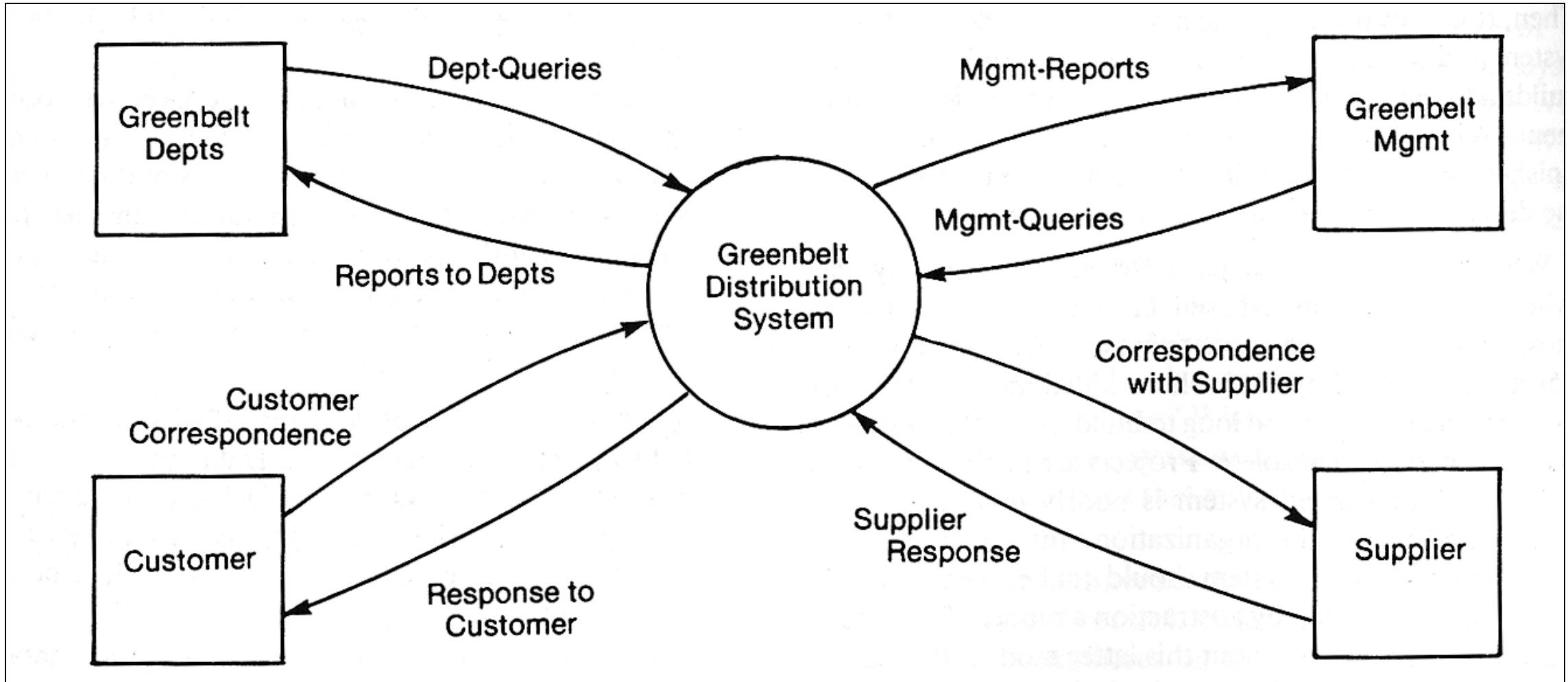
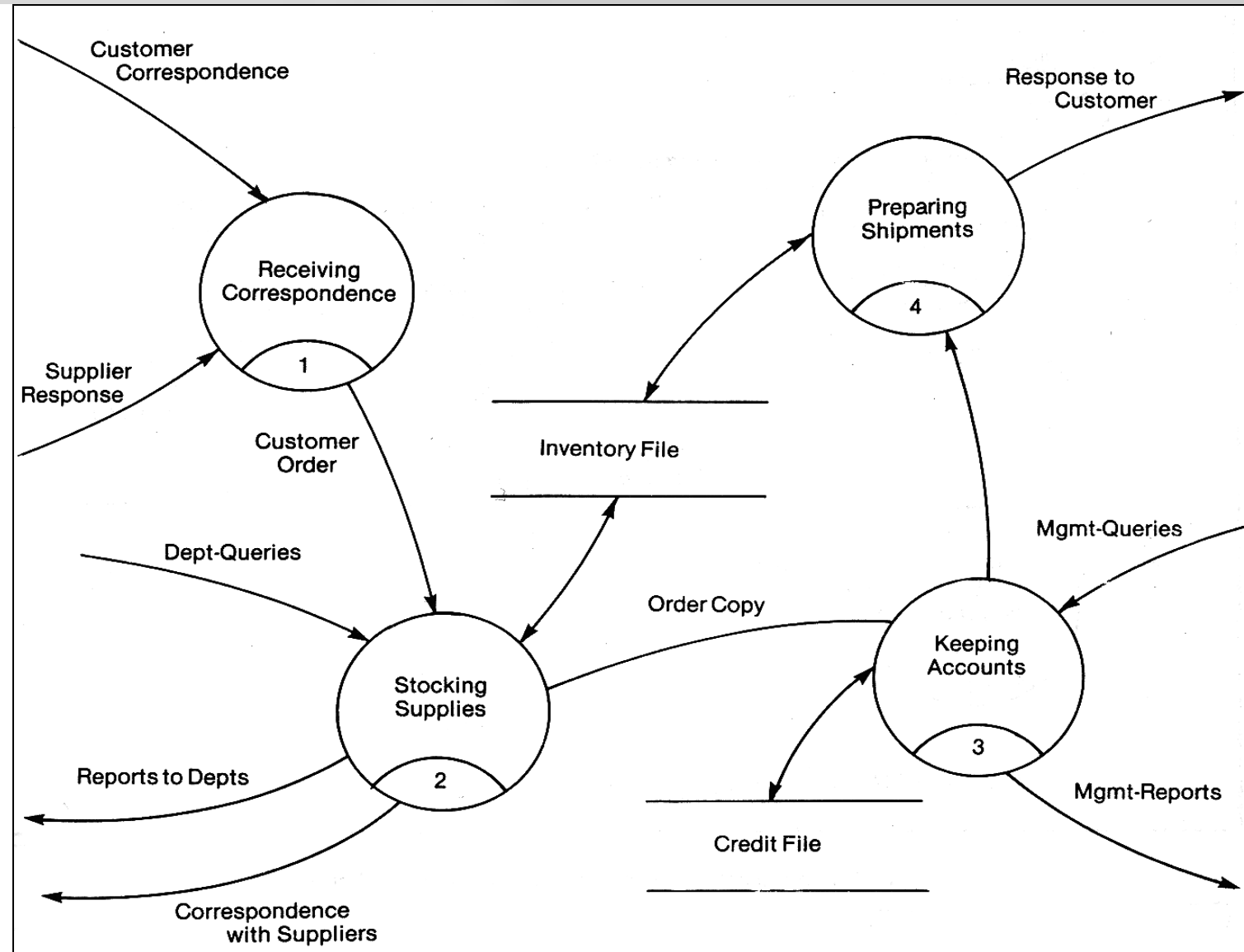


Figure 1: Context level data flow diagram

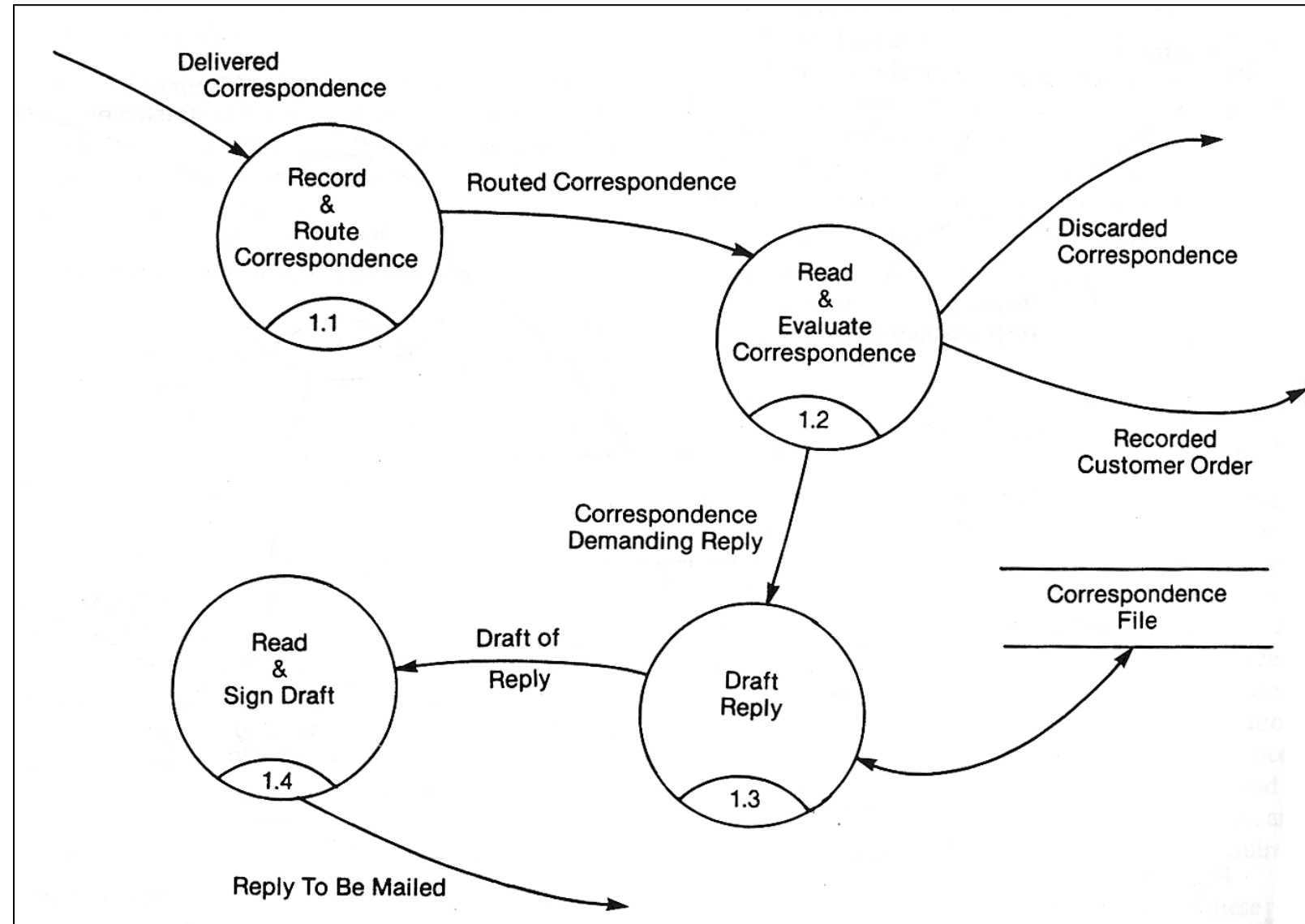
DFD – Data Process Interactions ...

Figure 2: Level 0 data flow diagram



DFD – Data Process Interactions ...

Figure 3:
Intermediate
level DFD:
Receiving
correspondence



DFD – Data Process Interactions

- Eventually the analyst reaches a level of detail at which it is no longer useful to decompose the functions and the data with which they interact.
- This level, called the "functional primitive"

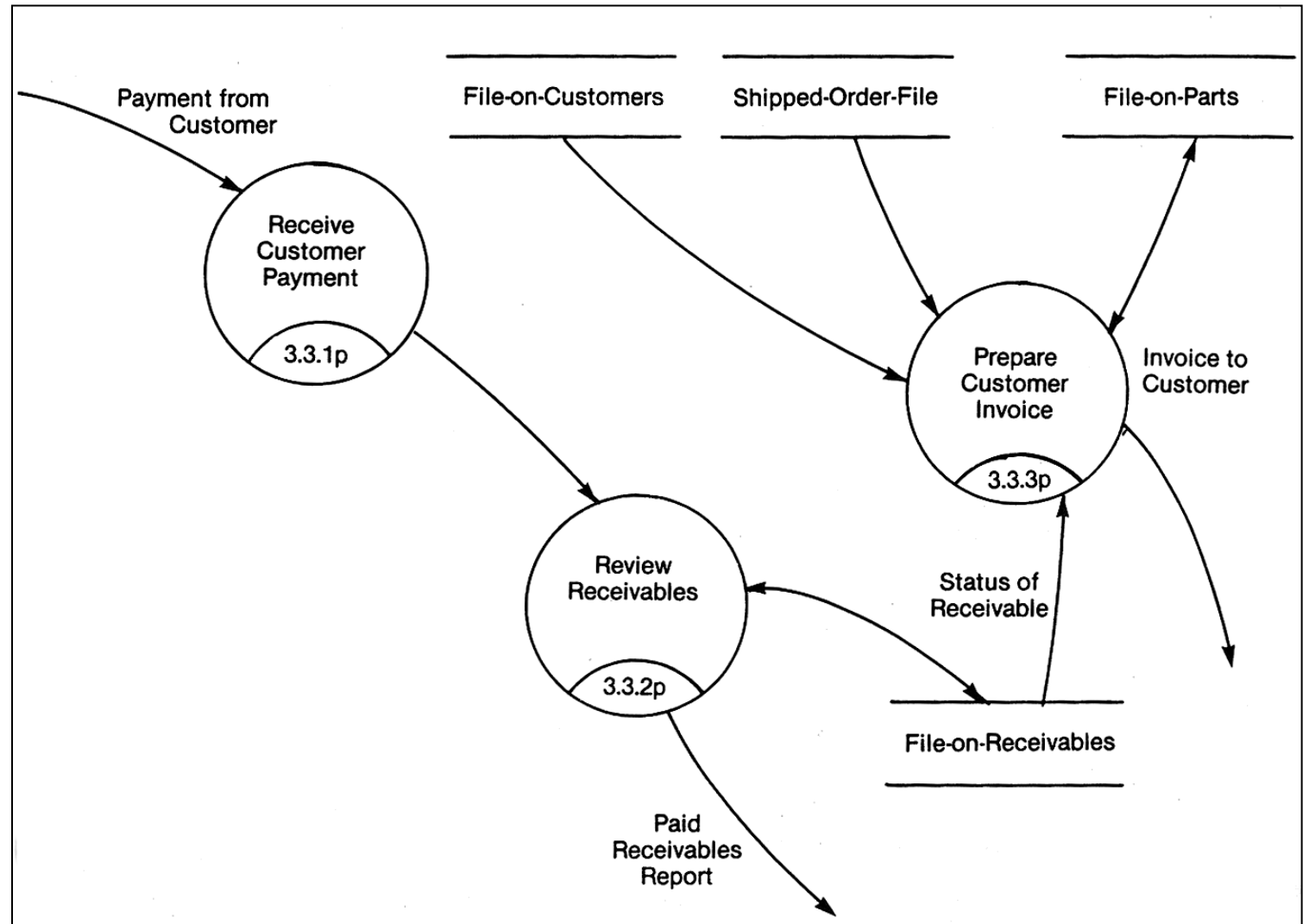
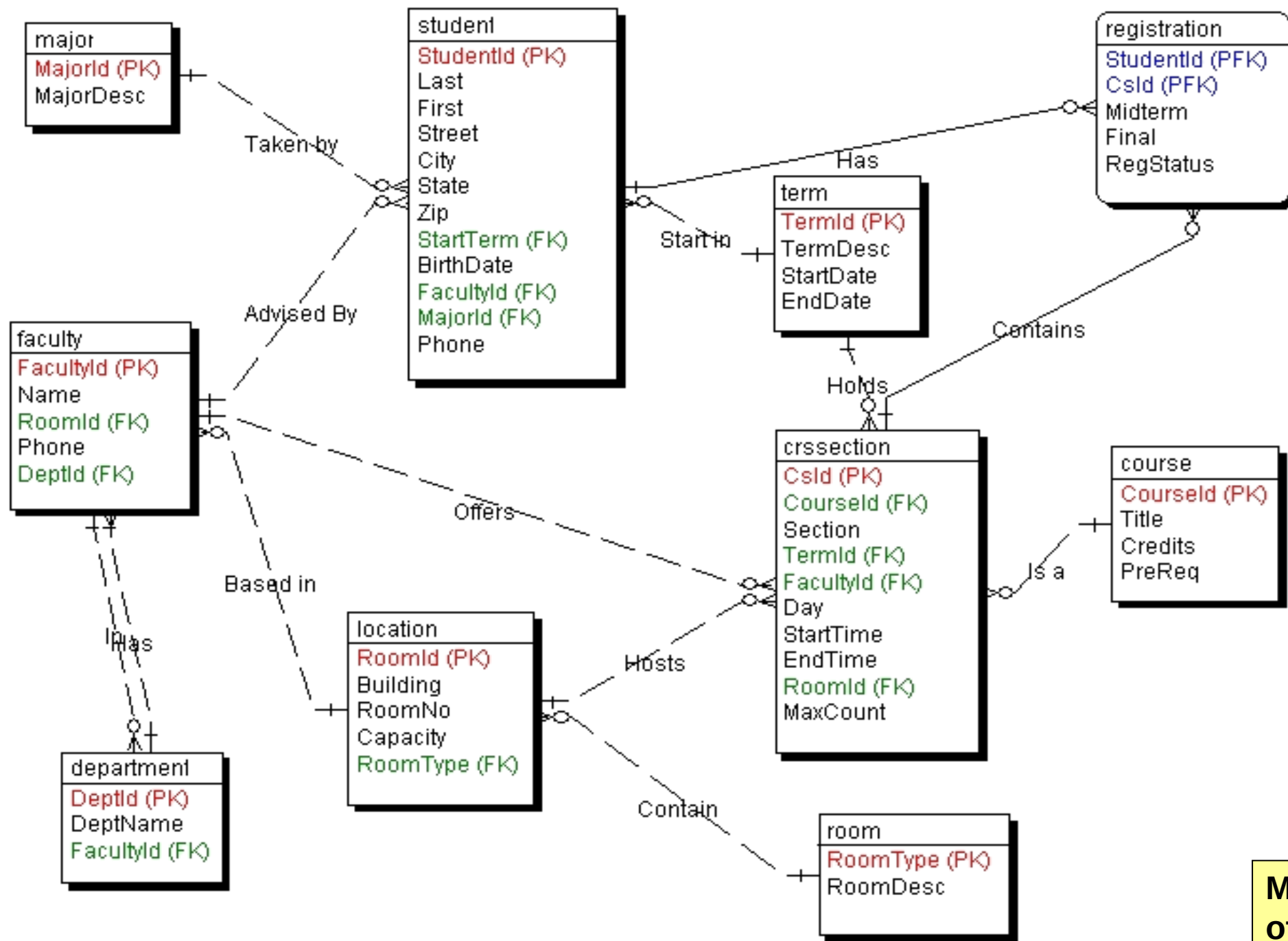


Figure 4: Functional primitive DFD: Accounts receivable

ERD Data Modeling

- The primary goal of ERD data modeling is to visualize the data requirements of all or part of an enterprise.
- The requirements include:
 - Things with which the business deals, called **entity types**.
 - Associations between these things, called **relationships**
 - Characteristics that describe each of these things, called **attribute entity types**.

ERD Data Modeling - example



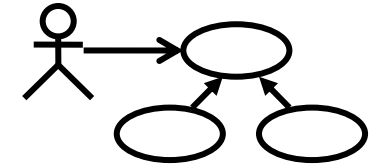
More on ERDs in other CS Courses

Object-Oriented Approach

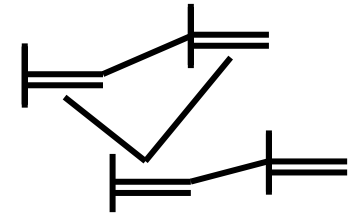
- Popular since 1990s
- **Data-centric** – it evolves around class models
 - But the growing significance of *use cases* in UML shifts the emphasis slightly from data to processes
 - **UML** (Unified Modeling Language), the notation we use throughout this course, is an **object-oriented notation** for software modeling.
- Matches the **event-driven** programming demanded by interactive GUI-based applications

UML First Pass

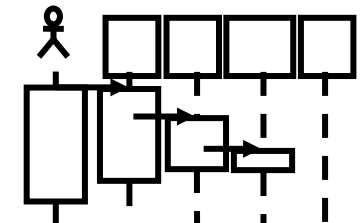
- Use case diagrams
 - Describe the functional behavior of the system as seen by the user
- Class diagrams
 - Describe the static structure of the system: Objects, attributes, associations
- Sequence diagrams
 - Describe the dynamic behavior between objects of the system
- StateChart diagrams
 - Describe the dynamic behavior of an individual object



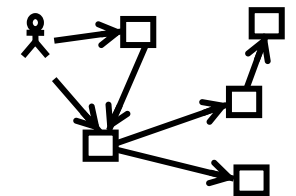
Use-Case Model



Class Diagrams



Sequence Diagrams

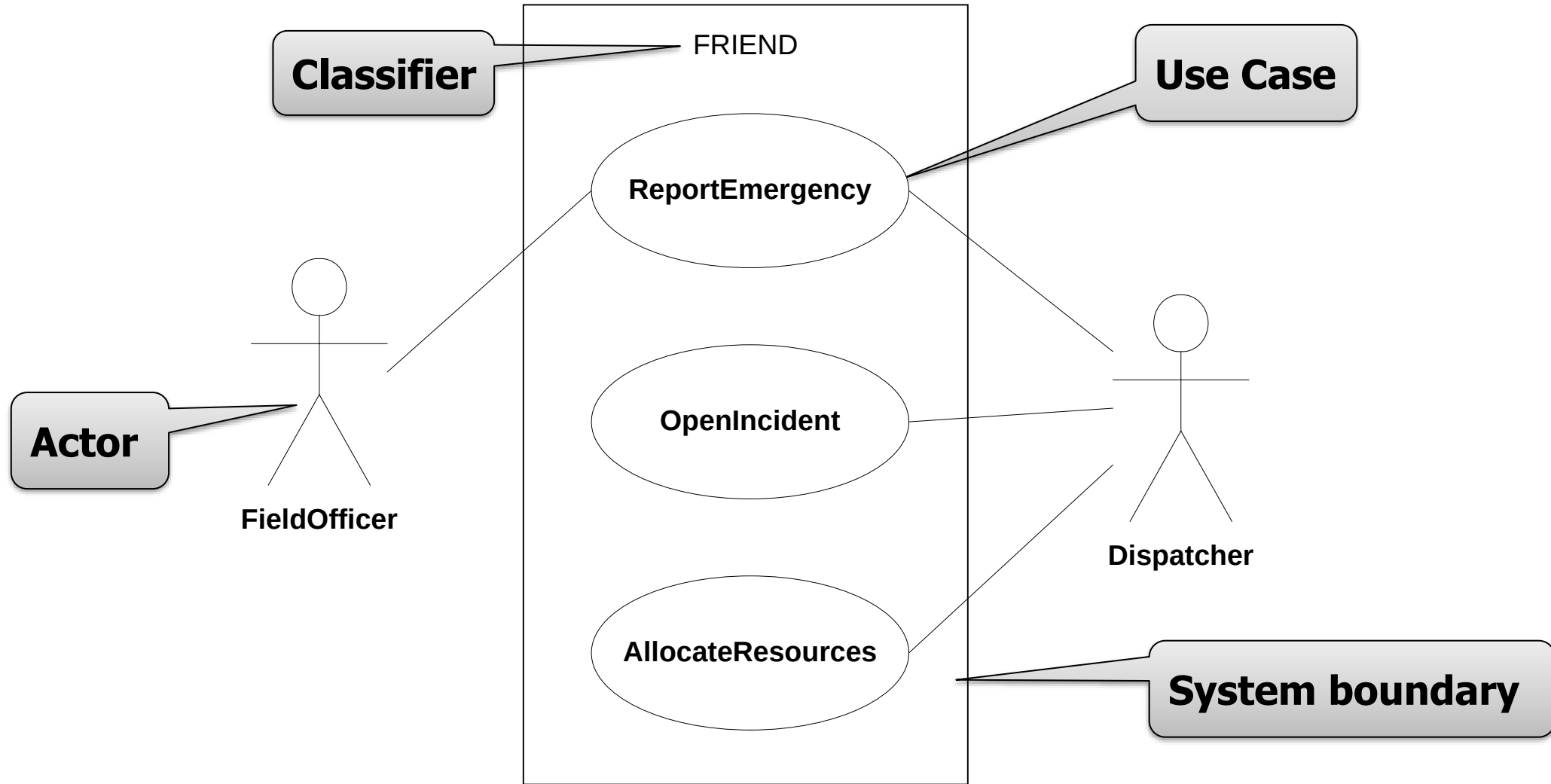


Communication Diagrams

UML Core Conventions

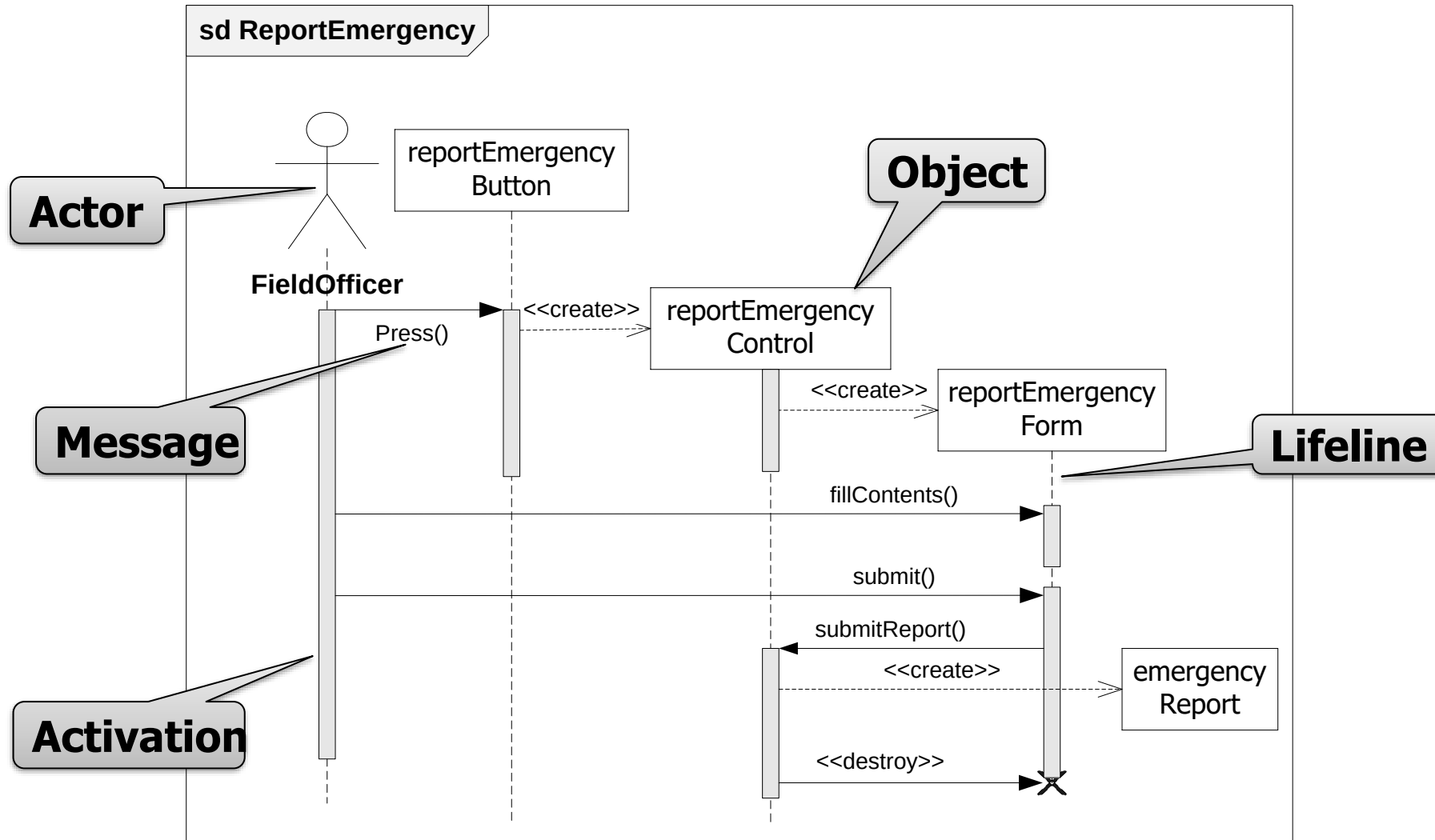
- All UML Diagrams denote graphs of nodes and edges
 - Nodes are entities and drawn as rectangles or ovals
 - Rectangles denote classes or instances
 - Ovals denote functions
- Names of Classes are not underlined
 - SimpleWatch
 - Firefighter
- Names of Instances are underlined
 - myWatch:SimpleWatch
 - Joe:Firefighter
- An edge between two nodes denotes a relationship between the corresponding entities

UML first pass: Use case diagrams



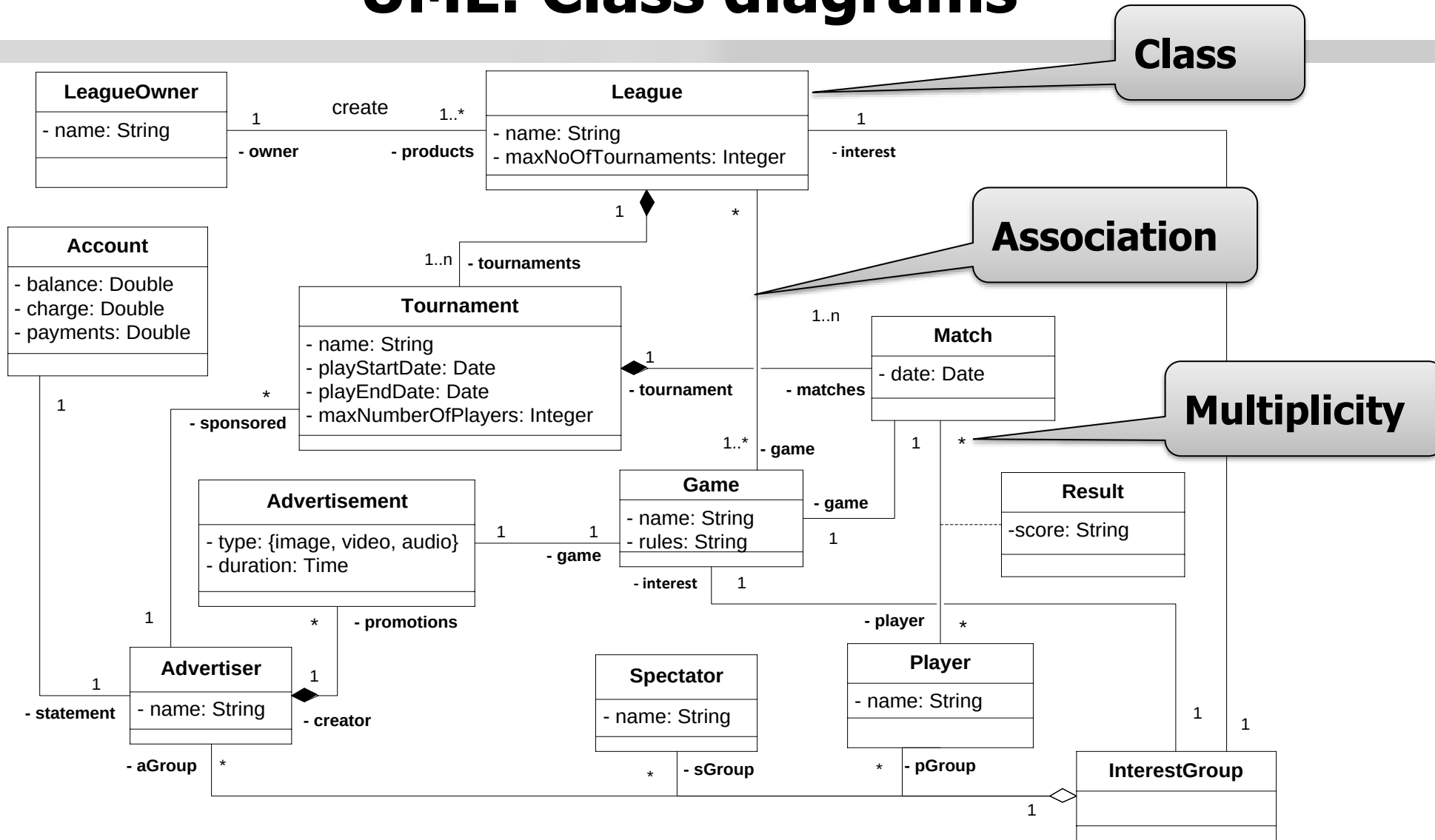
Use case diagrams represent the functionality of the system from user's point of view

UML: Sequence diagram



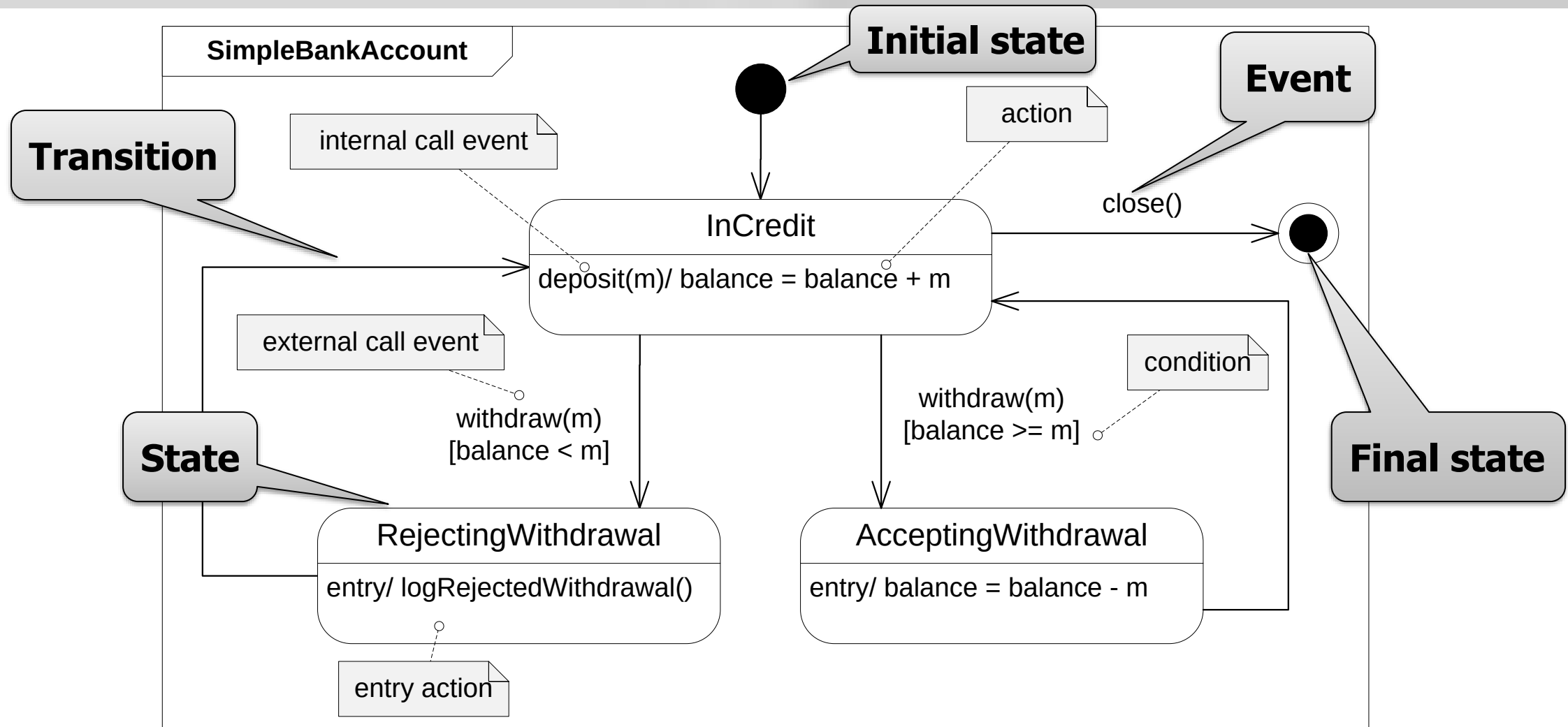
Sequence diagrams represent the behavior of a system

UML: Class diagrams



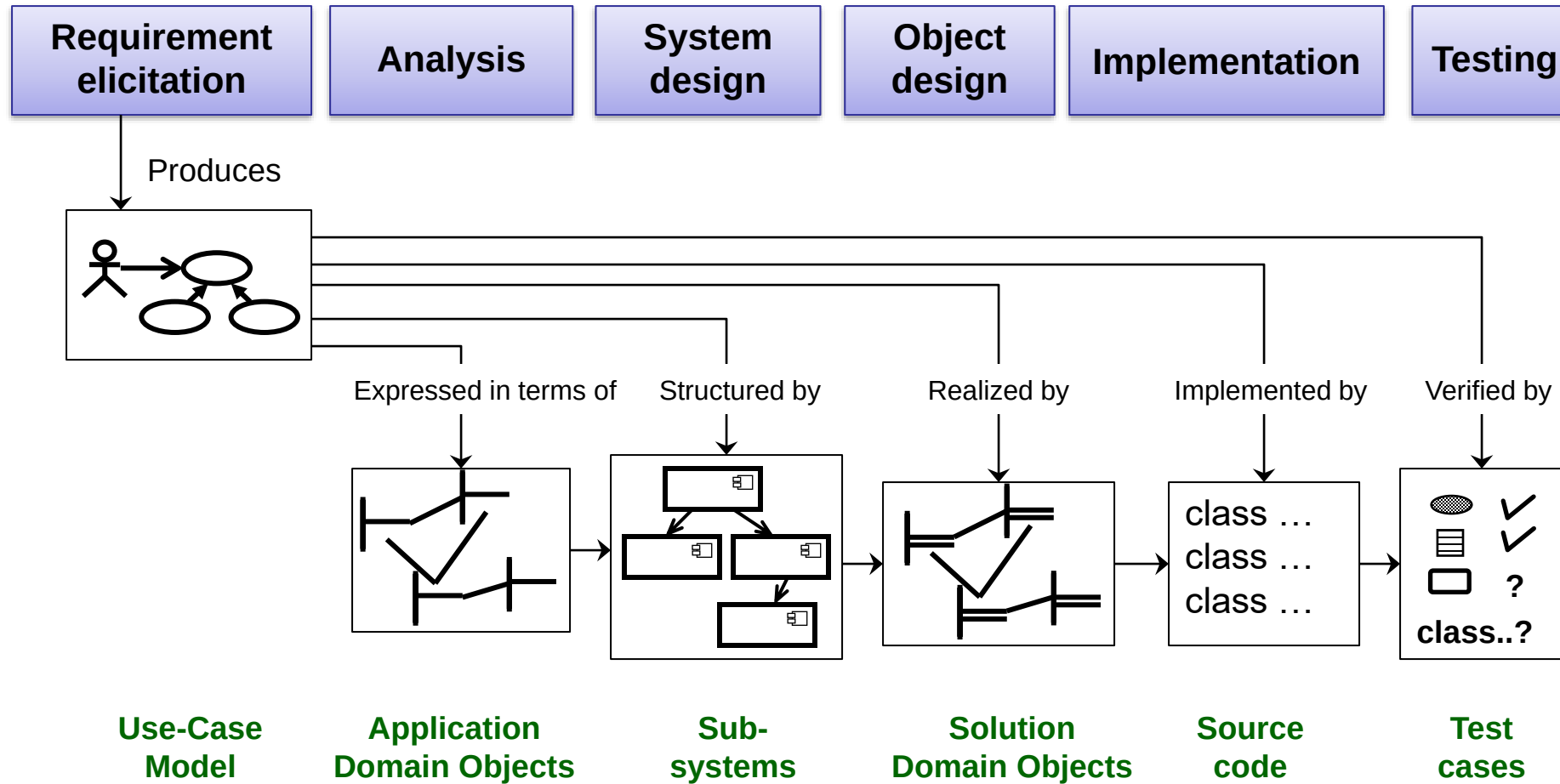
Class diagrams represent the structure of the system

UML: Statechart diagrams



Represent behavior of *a single object* with interesting dynamic behavior.

SDLC- Modeling



Software Development LifeCycle (SDLC)

- **Software development lifecycle** identifies:
 - the exact **activities** of developments from initial inception to phasing it out
 - the applied **modeling approach**
 - the **method** (methodology) and associated development process

SDLC - Methods

- A **method** is a repeatable technique for solving a specific problem.
 - A recipe is a method for cooking a specific dish.
 - A sorting algorithm is a method for ordering elements of a list.
- A **software development methodology** or **system development methodology** in software engineering is a framework that is used to structure, plan, and control the process of developing an information system.
- Every software development methodology framework acts as a basis for applying specific approaches to develop and maintain software.

SDLC - Development Methodologies

- Software life cycle models describe activities (phases) of the software cycle and **the order** in which those activities are executed.
- There are many different models, and many companies adopt their own, but all have very similar patterns. The following are some of these models:
 - Structured methodology
 - Waterfall model
 - V-Model
 - Rapid application development **RAD** methodology
 - Incremental and Iterative Model (Phased model)
 - Prototype model
 - Spiral Model
 - Agile software development methodology
 - Extreme Programming model
 - SCRUM

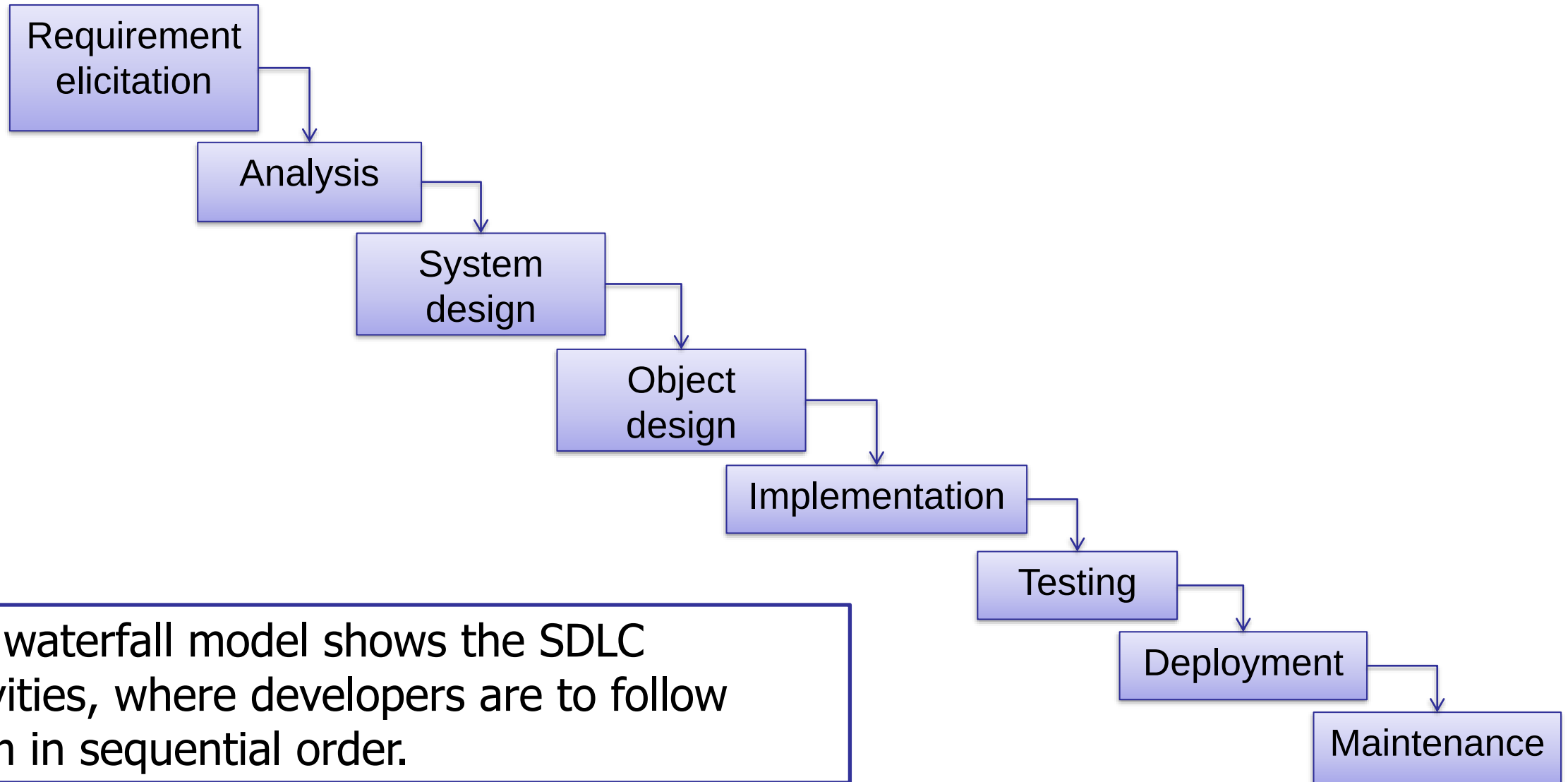
Waterfall model

of the Software Development Life Cycle (SDLC)

Waterfall model of the SDLC

- The waterfall model was first described by Royse in 1970
- It is the most common and classic of life cycle models.
- AKA a linear-sequential life cycle model.
- Waterfall-based software life cycle models are still widely used.
- In a waterfall model, each activity must be **completed** in its entirety before the next activity can begin, and activities **do not overlap**.
- At the end of each activity, a review takes place to determine whether or not to continue to the next activity.

Waterfall Model of the SDLC



Waterfall Model of the SDLC

- **Advantages**

- Simple and easy to use.
- Easy to manage – each activity has specific deliverables and a review process.
- Activities are processed and completed one at a time.

Waterfall Model of the SDLC ...

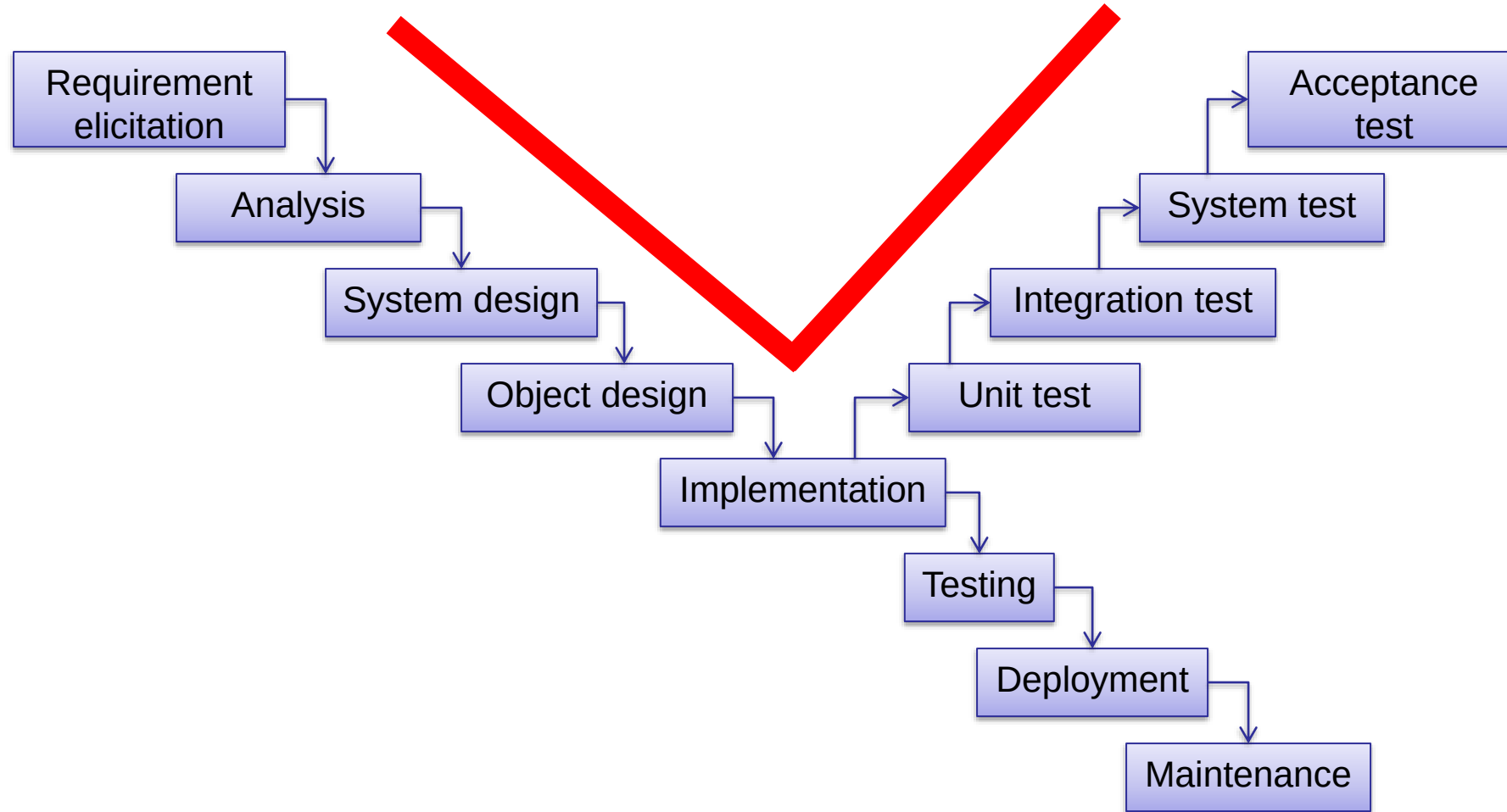
- **Disadvantages**

- It's difficult to respond to changing customer requirements.
- Adjusting scope during the life cycle can kill a project
- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Poor model for long run projects

The V-Model

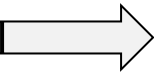
of the Software Development Life Cycle (SDLC)

From the Waterfall Model to the V-Model



From the Waterfall Model to the V-Model

- Like the waterfall model, the V-Shaped life cycle is a sequential path of execution of processes.
- Each activity must be completed before the next phase begins.
- The testing procedures are developed early in the life cycle before any coding begins in the implementation activity.
- Requirements begin the life cycle model and before analysis is started, an acceptance test plan is created.
- A system test plan is created during the analysis activities.



From the Waterfall Model to the V-Model ...

- System design activity focuses on system architecture and decomposition. An integration test plan is created in this activity in order to test the pieces of the software systems ability to work together.
- The object design activity is where the actual software components are designed, and unit tests are created in this phase as well.
- The implementation phase is, again, where all coding takes place. Once coding is complete, the path of execution continues up the right side of the V where the test plans developed earlier are now put to use.

V-Model of the SDLC

- **Advantages**

- Simple and easy to use.
- Each phase has specific deliverables.
- **Higher chance of success** over the waterfall model due to the development of test plans early during the life cycle.
- Works well **for small projects** where requirements are **easily understood**

V-Model of the SDLC ...

- **Disadvantages**

- Very rigid, like the waterfall model.
- Little flexibility and adjusting scope is difficult and expensive.
- Software is developed during the implementation phase, so **no early prototypes** of the software are produced.
- Model doesn't provide a clear path for problems found during testing phases.

SDLC - Development Methodologies

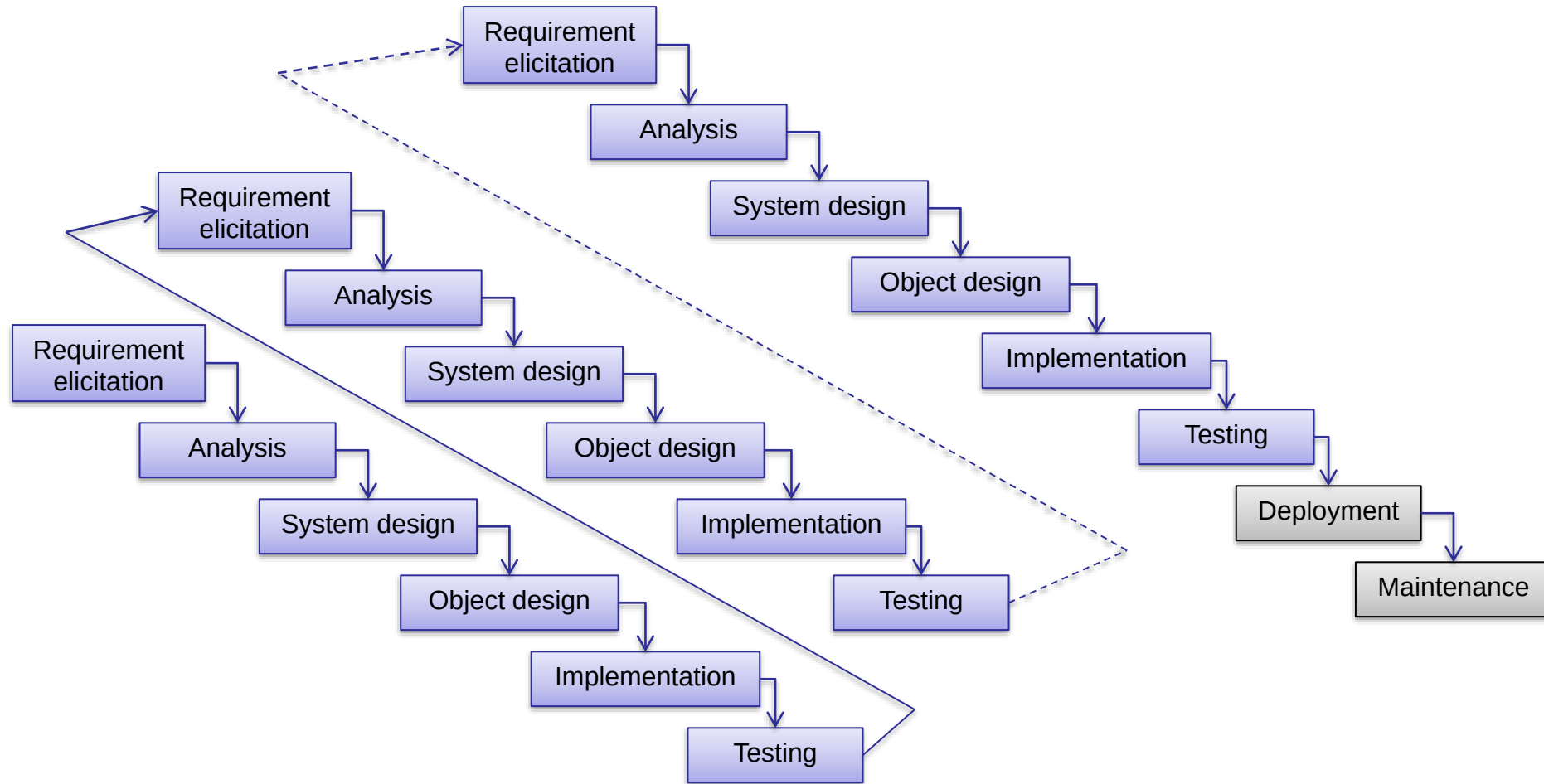
- Software life cycle models describe activities (phases) of the software cycle and **the order** in which those activities are executed.
- There are many different models, and many companies adopt their own, but all have very similar patterns. The following are some of these models:
 - Structured methodology
 - Waterfall model
 - V-Model
 - Rapid application development **RAD** methodology
 - Incremental and Iterative Model (Phased model)
 - Prototype model
 - Spiral Model
 - Agile software development methodology
 - Extreme Programming model
 - SCRUM

Incremental and Iterative model of the Software Development Life Cycle (SDLC)

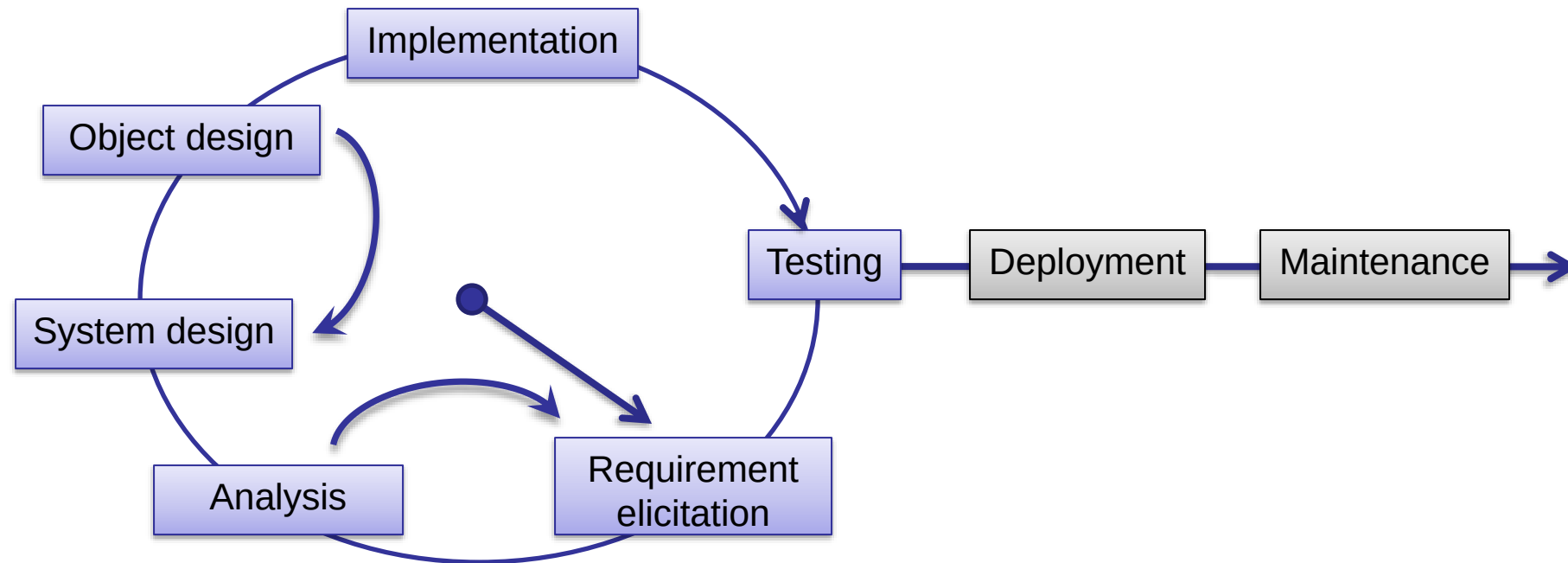
Incremental and Iterative (Phased) model

- Iterative and Incremental development is at the heart of a cyclic software development process developed in response to the weaknesses of the waterfall model.
- Each iteration passes through the requirements, design, implementation and testing activities.
- **A working version of software** is produced during the first iteration, so you have working software early on during the software life cycle.
- Subsequent iterations build on the initial software produced during the first iteration.

Incremental and Iterative (Phased) model



Incremental and Iterative (Phased) model – different drawing



- Iterative methodology allows us to iterate over the activities, moving backwards and forwards, or round and round.
- Each cycle results in **an executable release**. Perhaps after 3 or 4 cycles, the system is complete, then deployment can be performed. ►

Incremental and Iterative model

- **Advantages**

- System functionality is available earlier and customer does not have to wait as long.
- Early increments act as a prototype to help elicit requirements for later increments.
- The highest priority functionalities tend to receive more testing.
- More flexible – less costly to change scope and requirements.
- Easier to test and debug during a smaller iteration.
- Each iteration is an easily managed milestone.

Incremental and Iterative model

- **Disadvantages**

- Each activity of an iteration is rigid and do not overlap each other.
- **Problems may arise relating to system architecture** because not all requirements are gathered up front for the entire software life cycle.

Prototype model

of the Software Development Life Cycle (SDLC)

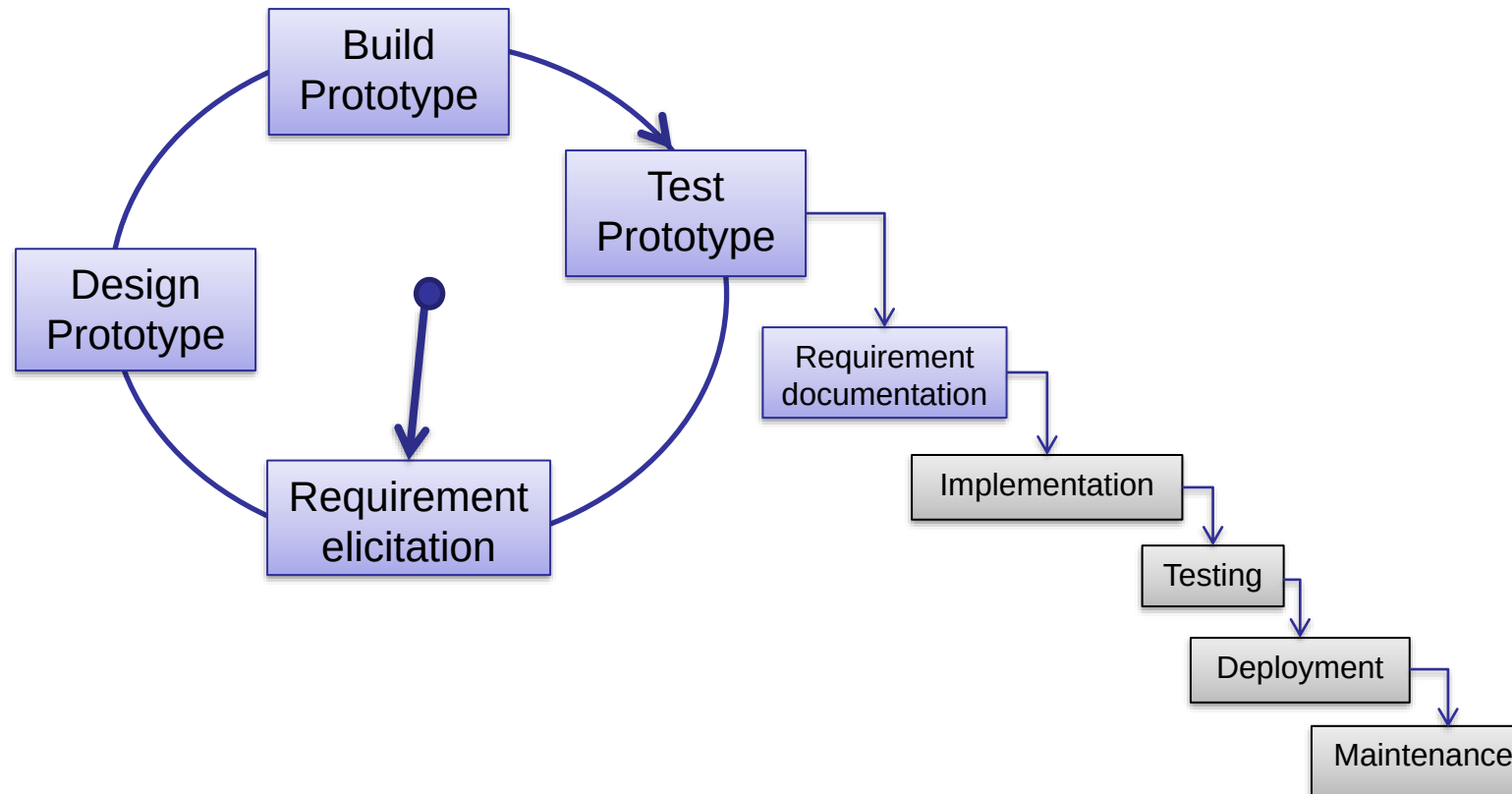
Prototype model

- A prototype is built, tested, and then reworked as necessary until **an acceptable requirements** is finally achieved from which the complete system can now be developed.
 - Prototype paradigm begins with requirements elicitation activity. A quick design occurs which leads to the construction of prototype.
 - Prototype is evaluated by the customer/user and used to refine the requirements for the software to be developed.
 - Iteration occurs as the prototype is tuned to satisfy the user requirements.
 - Two kinds: 1. **System** prototype 2. **Throw-away** prototype

System prototype

- **System** prototype
 - targets the speed of product delivery
 - retained after the requirements elicitation process and used to produce the final product.
 - typically **concentrates on well-understood requirements** so that the **first version of the product can be delivered quickly** (although with incomplete functionality).

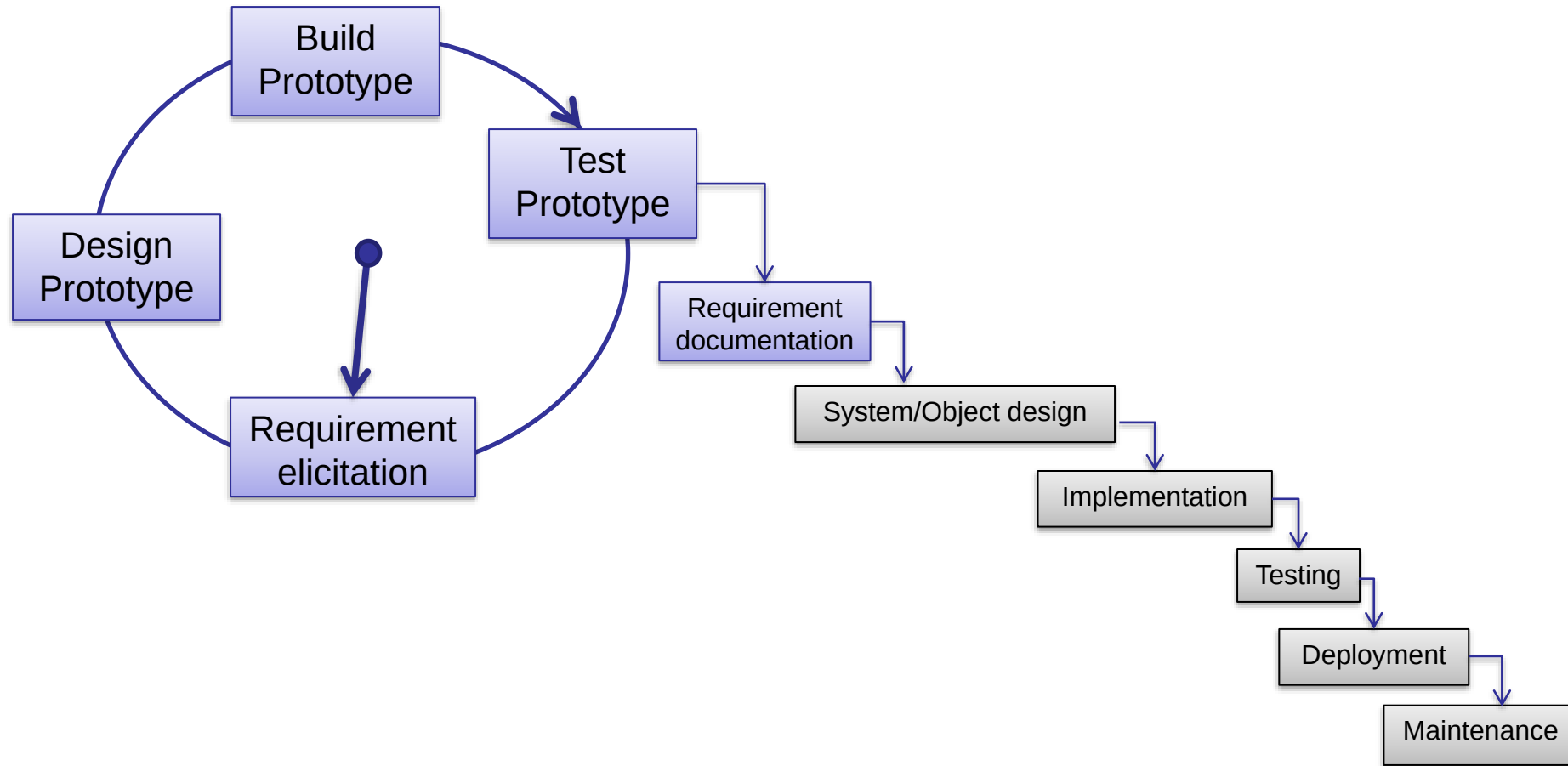
System Prototype Model of the SDLC



Throw-away prototype

- **Throw-away** prototype
 - targets requirement determination phase of the lifecycle
 - discarded when the requirements elicitation is completed
 - typically concentrates **on the least understood requirements**

Throw-away Prototype Model of the SDLC



Prototype Model of the SDLC

- **Advantages**

- Help understand the requirements for the user interface
- Examining feasibility of a proposed design approach
- Exploring system performance issues

- **Disadvantages**

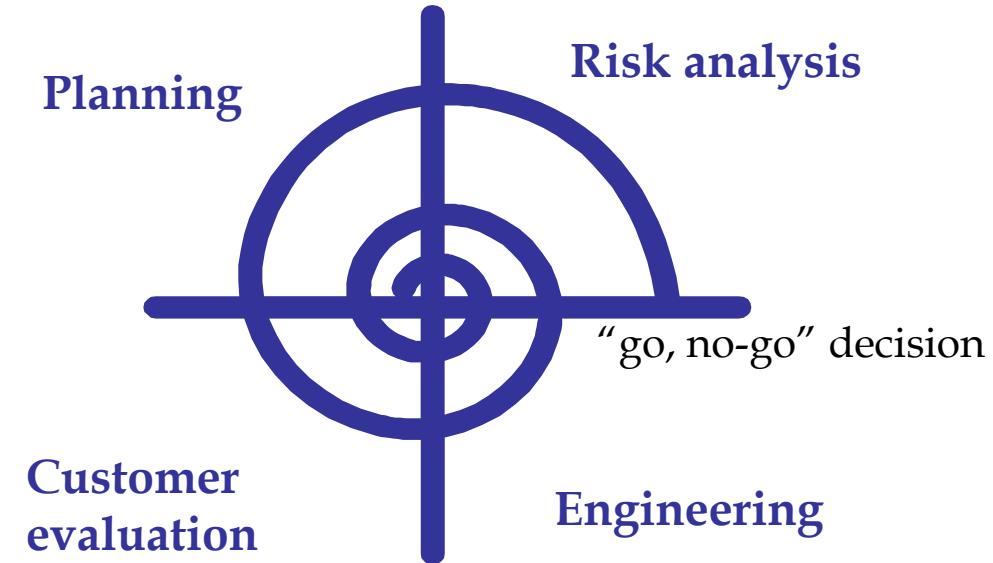
- Users treat the prototype as the solution
- A prototype is only a partial specification

Spiral model

of the Software Development Life Cycle (SDLC)

Spiral Model – SDLC

- The spiral model places the SDLC activities within the wider context of **system planning, risk analysis, building, and customer evaluation**.
- These four phases are graphically visualized as quadrants, which together create spiral loops on a Cartesian diagram. Boehm's spiral model 1987.
- It based on the same activities as the waterfall model; however, it adds several activities such as **risk management** and **prototyping** to each activity.

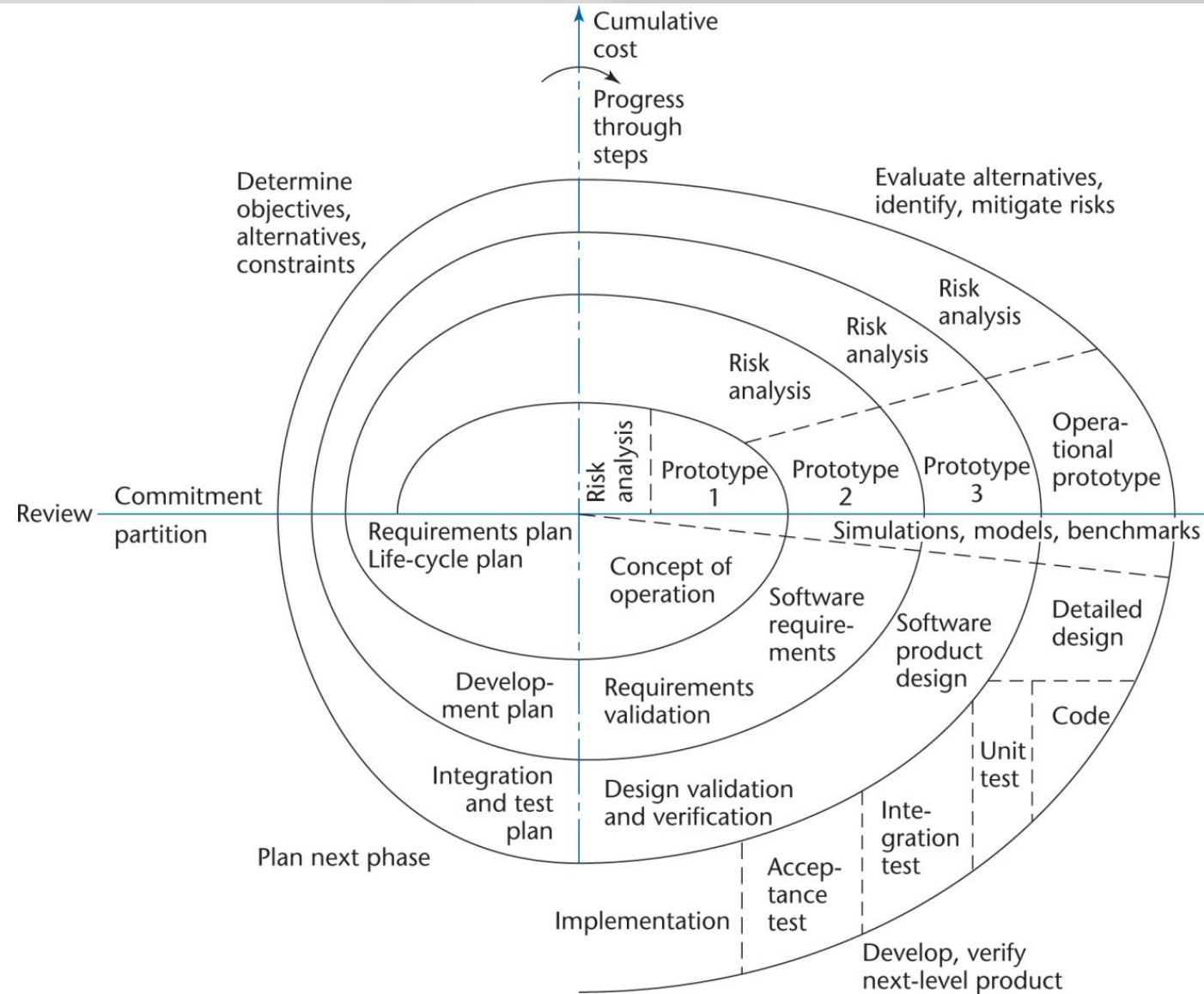


Spiral Model – SDLC ...

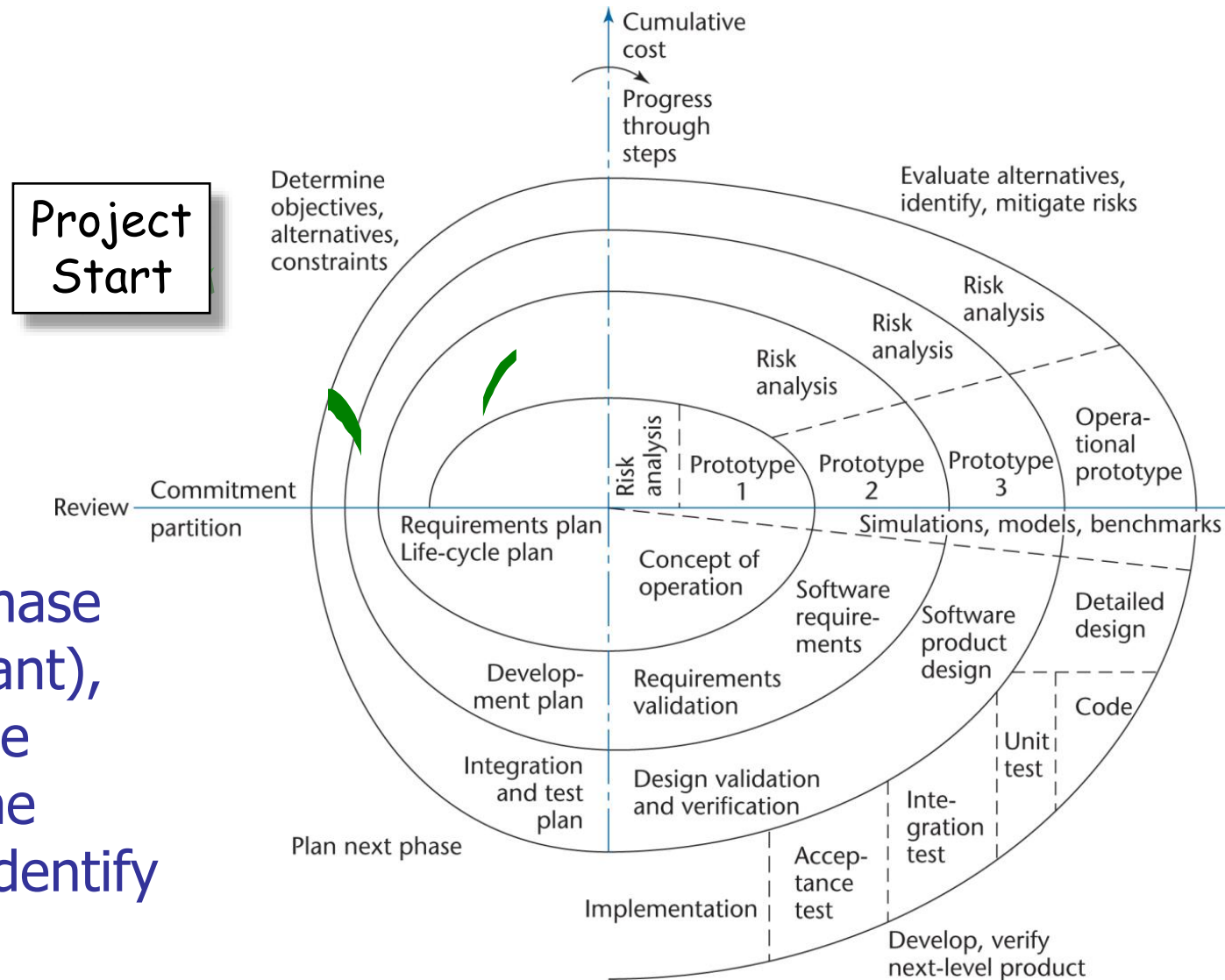
- Spiral model has the following set of activities and applied in rounds:
 - Determine objectives and constraints
 - Evaluate alternatives
 - Identify risks
 - Resolve risks by assigning priorities to risks
 - Develop a series of prototypes for the identified risks starting with the highest risk
 - **Use a waterfall model for each prototype development**
 - If a risk has successfully been resolved, evaluate the results of the round and plan the next round
 - If a certain risk cannot be resolved, terminate the project immediately

Spiral Model – SDLC ...

Each round
is composed
of four
phases

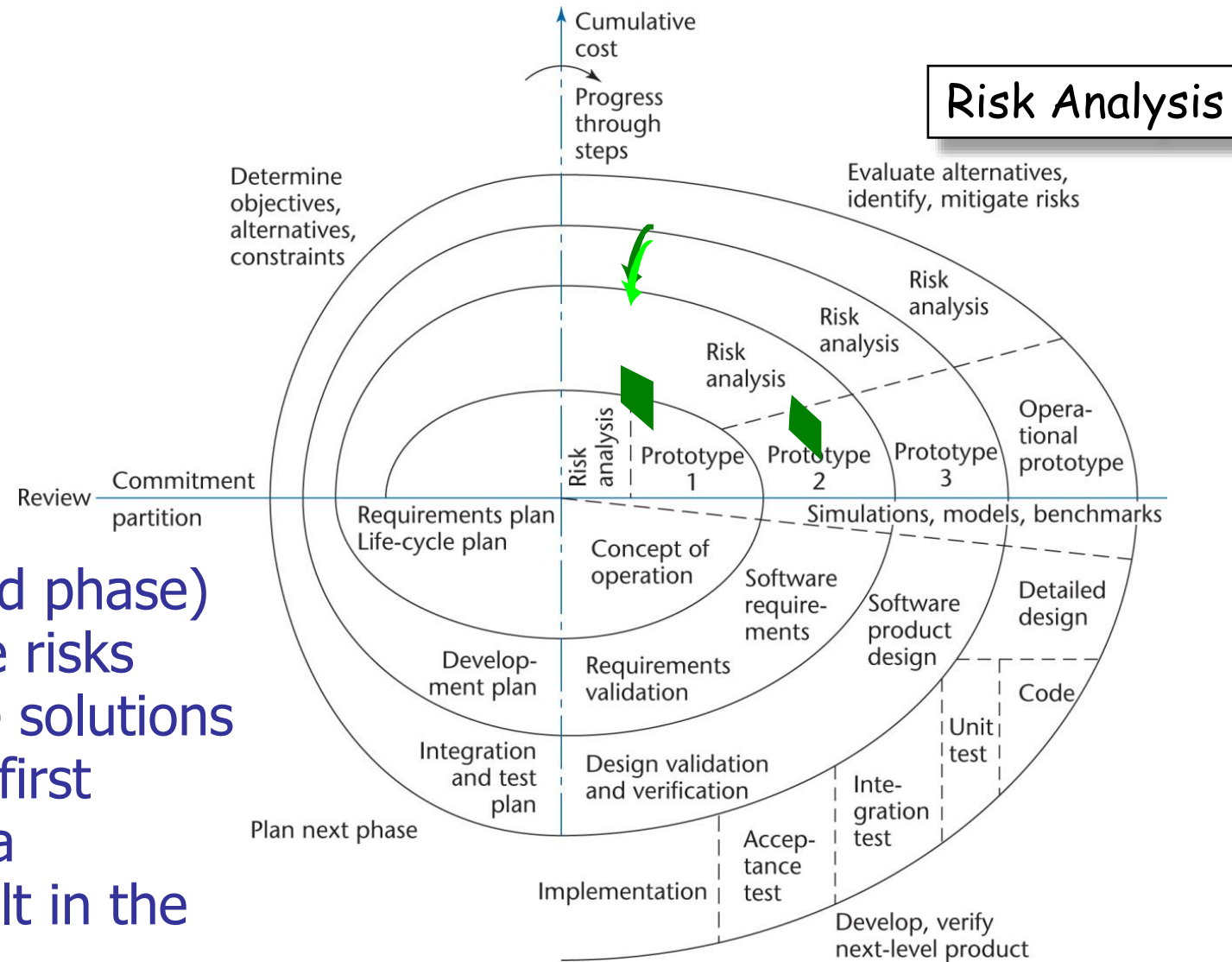


Round 1- Concept of Operations



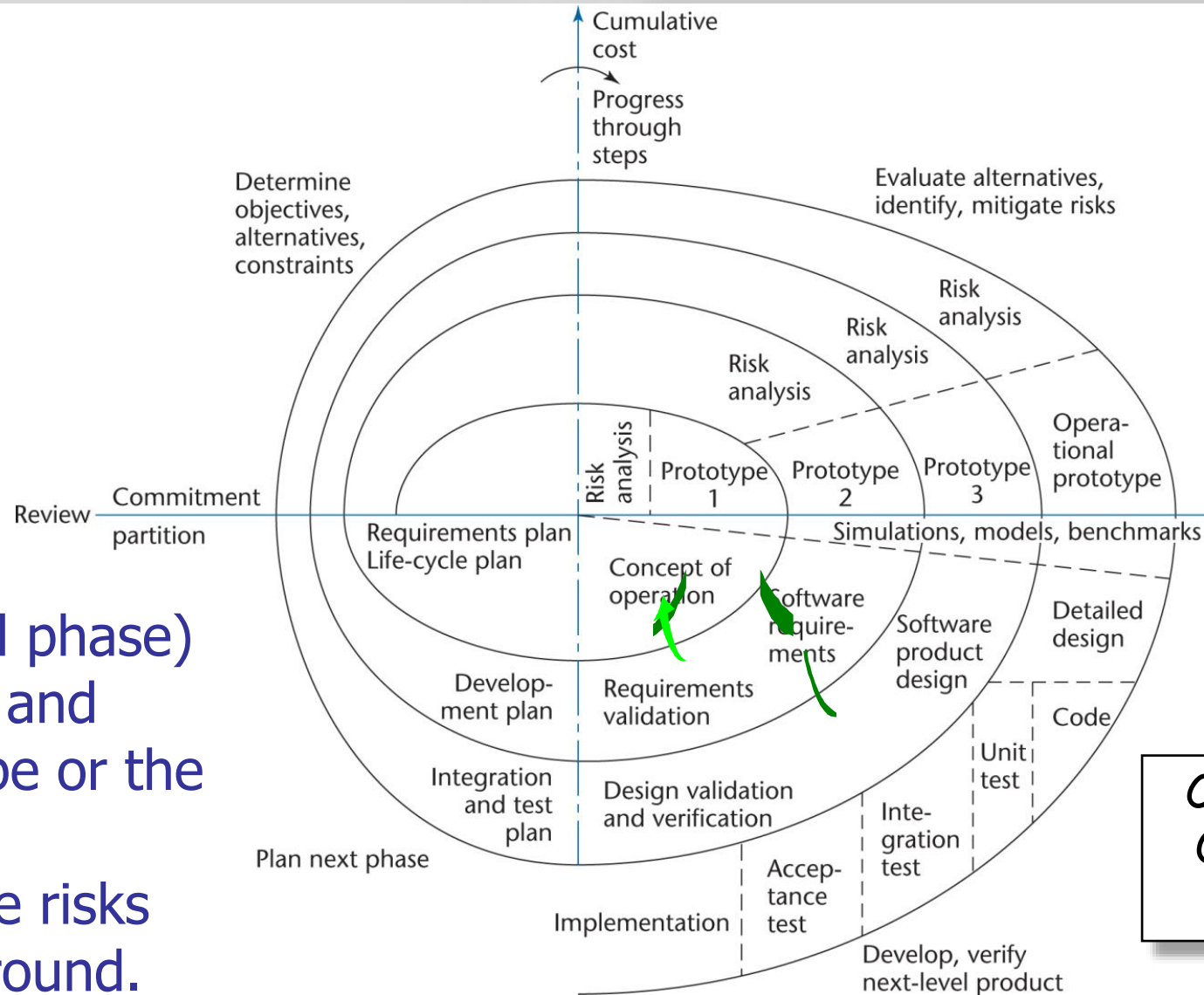
During the first phase (upper left quadrant), developers explore alternatives, define constraints, and identify objectives.

Round 1- Concept of Operations



Quadrant I: (second phase) developers manage risks associated with the solutions defined during the first phase and specify a prototype to be built in the next phase

Round 1- Concept of Operations

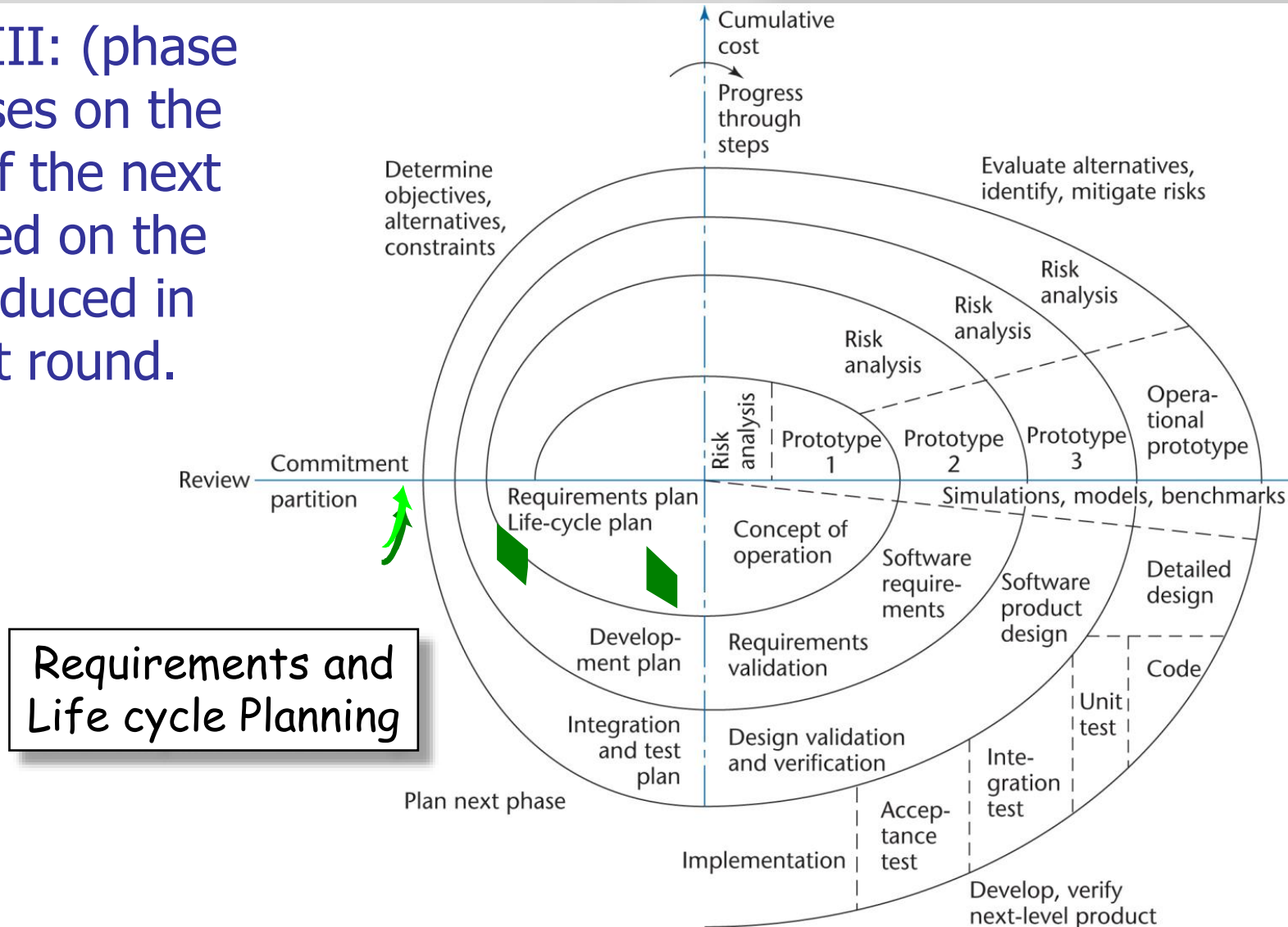


Quadrant II: (third phase) developers realize and validate a prototype or the part of the system associated with the risks addressed in this round.

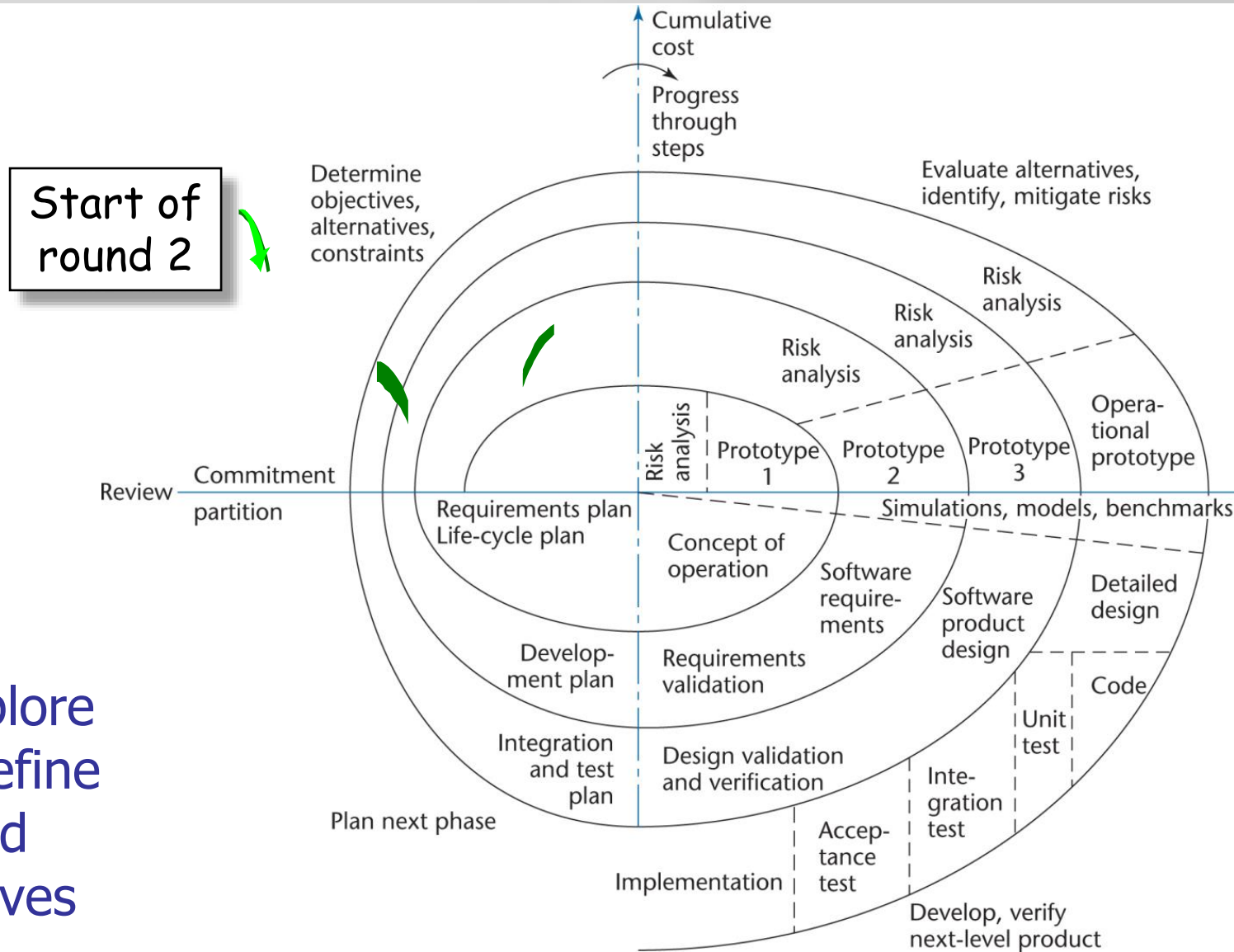
Concept of
Operation
Activity

Round 1- Concept of Operations

Quadrant III: (phase four) focuses on the planning of the next round based on the results produced in the current round.



Round 2, Software Requirements



Quadrant IV:
developers explore
alternatives, define
constraints, and
identify objectives

Spiral Model - SDLC

- **Advantages**

- The emphasis on alternatives and constraints supports the reuse of existing software.
- Specifically, the spiral model is applicable to large scale software project.

- **Disadvantages**

- A major strength of the spiral model is that it is risk driven, but this can also be a weakness.
- If risk analysis leads to the conclusion that the project should be terminated, an attempt by either customer or developer to terminate their contract can lead to a breach-of-contract lawsuit.

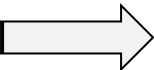
SDLC - Development Methodologies

- Software life cycle models describe activities (phases) of the software cycle and **the order** in which those activities are executed.
- There are many different models, and many companies adopt their own, but all have very similar patterns. The following are some of these models:
 - Structured methodology
 - Waterfall model
 - V-Model
 - Rapid application development **RAD** methodology
 - Incremental and Iterative Model (Phased model)
 - Prototype model
 - Spiral Model
 - Agile software development methodology
 - Extreme Programming model
 - SCRUM

Extreme Programming model of the Software Development Life Cycle (SDLC)

Agile Software Development Model

- Agile development is a group of programming-centric methodologies that focus on streamlining the SDLC.
- **Much of** the modeling and documentation overhead is eliminated; instead, face-to-face communication is preferred.
- A project emphasizes simple **iterative application development** in which every iteration is a complete software project, including all developing activities.
- Iterations are kept short (one to four weeks).



Agile Software Development Model ...

- There are several popular approaches to agile development, including extreme programming (XP), Scrum, and Dynamic Systems Development Method (DSDM)
- Here, we briefly describe eXtreme Programming and Scrum.

Dilbert on Extreme Programming



www.dilbert.com
scottadams@aol.com



1/9/03 © 2002 United Feature Syndicate, Inc.



www.dilbert.com
scottadams@aol.com



1/10/03 © 2002 United Feature Syndicate, Inc.



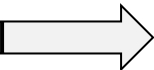
Agile development: **Extreme programming XP**

- XP is a software development methodology developed by **Kent Beck**.
- An XP project begins with user **stories** that features client wants
 - Estimate duration and cost of each story
 - The client selects the features to be included in each successive build using cost–benefit analysis
 - Each build is divided into **tasks**
 - A programmer first draws up test cases for a task; this is termed **test- driven development (TDD)**.
- For each task, two programmers work together on one computer (**pair programming**)
- Ideally, implementing and integrating a task should take no more than a few hours.

Unusual Features of XP

A number of features of extreme programming (XP) are somewhat different from the way in which software is usually developed:

1. The computers are put in the center of a large room lined with cubicles
2. A client representative works with the XP team at all times.
3. Software professionals cannot work overtime for 2 successive weeks
4. There is no specialization. Instead, all members of the XP team work on requirements, analysis, design, code, & testing.



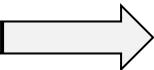
Unusual Features of XP

5. There is no overall design step before the various builds are constructed. Instead, the design is modified while the product is being built (**Refactoring**).
6. A principle of XP is to minimize the number of features; there is no need to build a product that does any more than what the client actually needs.

Extreme Programming - Agile Processes

Agile processes are a collection of new paradigms characterized by:

- Less emphasis on analysis and design than in almost all other modern life-cycle models.
- Implementation starts much earlier in the life cycle because working software is considered more important than detailed documentation.
- Responsiveness to changes in the client requirements, and so is the importance of collaborating with the client.



Extreme Programming - Agile Processes

- Deliver working software frequently, Ideally every 2 or 3 weeks
 - One way of achieving this is to use *timeboxing*
 - The team members then do the best job they can during that time
 - It gives the client confidence to know that a new version with additional functionality will arrive every 3 weeks.

Extreme Programming - Agile Processes

- Another common feature of agile processes is *stand-up meetings*
 - Short meetings held at a regular time each day
 - Participants stand in a circle, they do not sit around a table to ensure the meeting lasts no more than 15 minutes
- The aim of a stand-up meeting is to raise problems not solve them.
- Solutions are found at follow-up meetings, preferably held directly after the stand-up meeting

Extreme Programming - Agile Processes

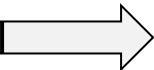
- At a stand-up meeting, each team member in turn answers 5 questions:
 - What have I done since yesterday's meeting?
 - What am I working on today?
 - What problems are preventing me from achieving this?
 - What have we forgotten?
 - What did I learn that I would like to share with the team?

Scrum model

of the Software Development Life Cycle (SDLC)

Scrum

- Scrum is a **simple management framework** for incremental product development using one or more **cross-functional, self organizing teams** of about **seven people each**.
- Scrum teams use fixed length iterations, called **Sprints**, typically two weeks or 30 days long. ◀
- Scrum teams attempt to build a potential shippable (properly tested) product increment every iteration.



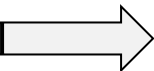
Scrum

- It is a **framework**, not a defined process or methodology. It provides a structure of **roles, meetings,** and **artifacts**. (more details later)
- Scrum teams are responsible for creating and adapting their processes within this framework.
- **Unlike XP, Scrum does not prescribe specific engineering practices.**

Scrum Roles (1)

- **Product Owner**

- The single individual responsible for return on investment (ROI) of the product development effort.
- Owns the product vision, re-prioritizes the Product Backlog, and revises release plan expectations.
- The final arbiter of requirements questions, including which items are considered “done” at the Sprint Review Meeting.



Scrum Roles (1) ...

• roles, meetings, and Artifacts

- **Scrum Development Team**

- Cross-functional group (about seven (+/- 2) individuals)
- Attempt to build a potential shippable product increment every sprint
- Collaborates and a self-organizing

Scrum Roles (2)

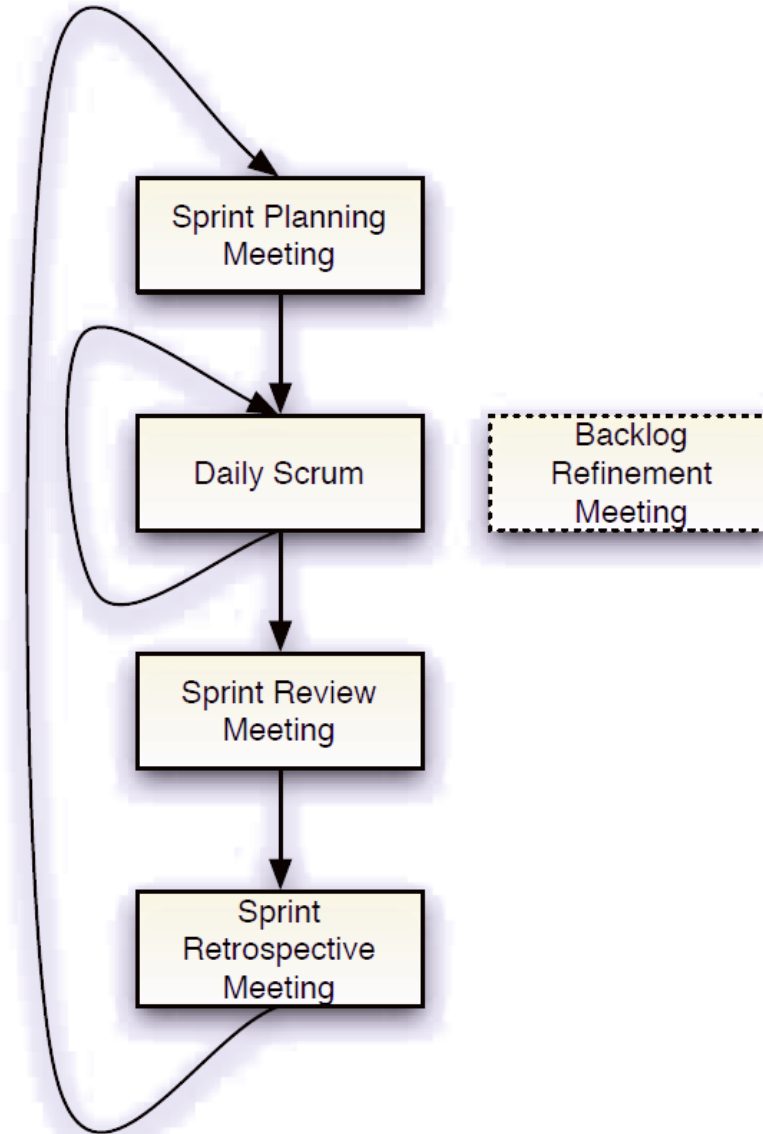
ScrumMaster

- facilitates the Scrum process, keeps the Scrum artifacts visible,
- facilitates Team self-organization (keeping it in the “zone”),
- shields the team from interference, and
- encourage the team to improve their engineering practices.
- ScrumMaster does these things (and more) **without any authority on the Team.**
- ScrumMaster does not make business or technical decisions.
- ScrumMaster is not the project manager.

Scrum Meetings (1)

• roles, meetings, and Artifacts

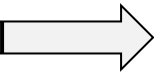
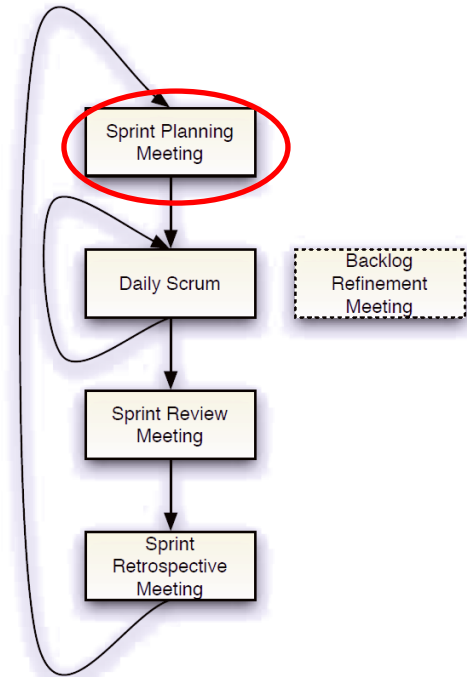
- All Scrum Meetings are facilitated by the ScrumMaster, though he/she has no decision-making authority at these meetings.



Scrum Meetings (2)

- **Sprint Planning Meeting**

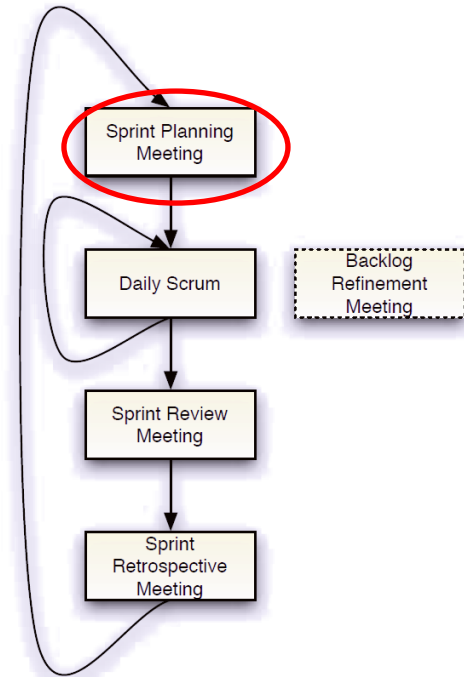
- **Part 1:** At the beginning of each iteration, the Product Owner and Team negotiate which Product Backlog Items for this Sprint.
 - The Product Owner is responsible for declaring which items are the most important to the business, and
 - The Team is responsible for committing to the amount of work they feel they can accomplish.



Scrum Meetings (2)

- **Sprint Planning Meeting**

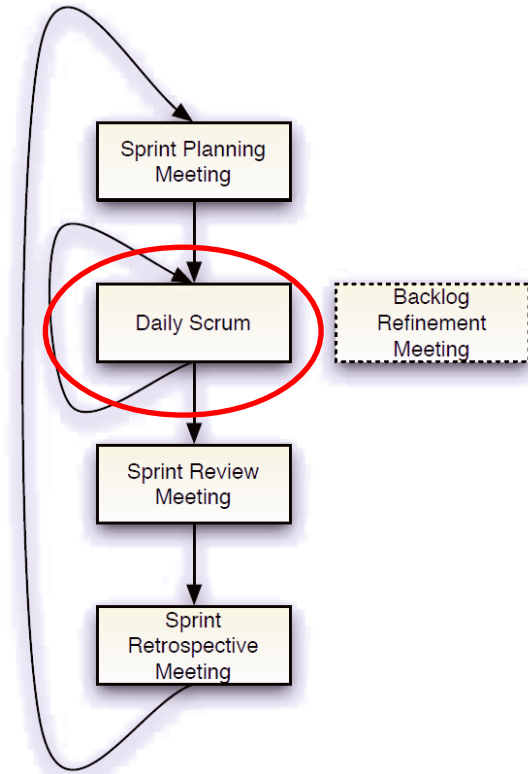
- **Part 2:** The Team decomposes the selected Product Backlog Items into an initial list of Sprint Tasks and makes a final commitment to do the work.
- The maximum time for planning a 30-day Sprint is 8 hours.



Scrum Meetings (2)

Daily Scrum Meeting

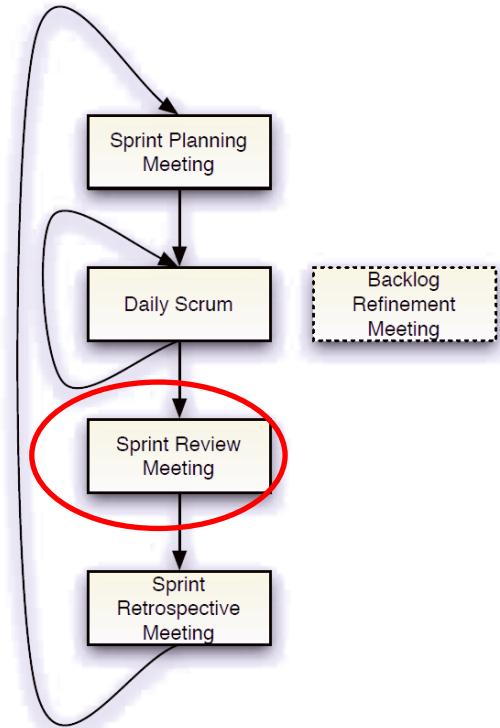
- Every day, at the same time and place, the Scrum Development Team members spend 15 minutes reporting to each other.
- It's a common practice to be done in a stand-up meeting format.
- Each team member reports to the rest of the team what s/he did the previous day, what s/he will do today, and what problems s/he has.
- The team will typically examine the current Sprint Task list, and Sprint Burndown Chart. (more details later)



Scrum Meetings (3)

Sprint Review Meeting

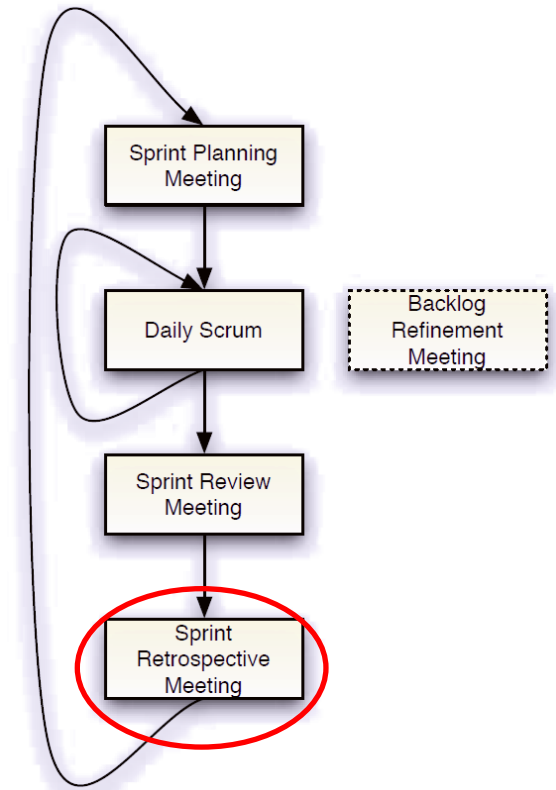
- At the end of each Sprint execution, the Team demonstrates the actual working product increment they built to the Product Owner.
- Incomplete items are returned to the Product Backlog candidates for future Sprints.
- The Product Owner declares which committed items will be considered “done” according to the previously negotiated agreement.
- The Sprint Review Meeting is an opportunity to iteratively refine the understanding of the requirements.



Scrum Meetings (4)

Sprint Retrospective Meeting

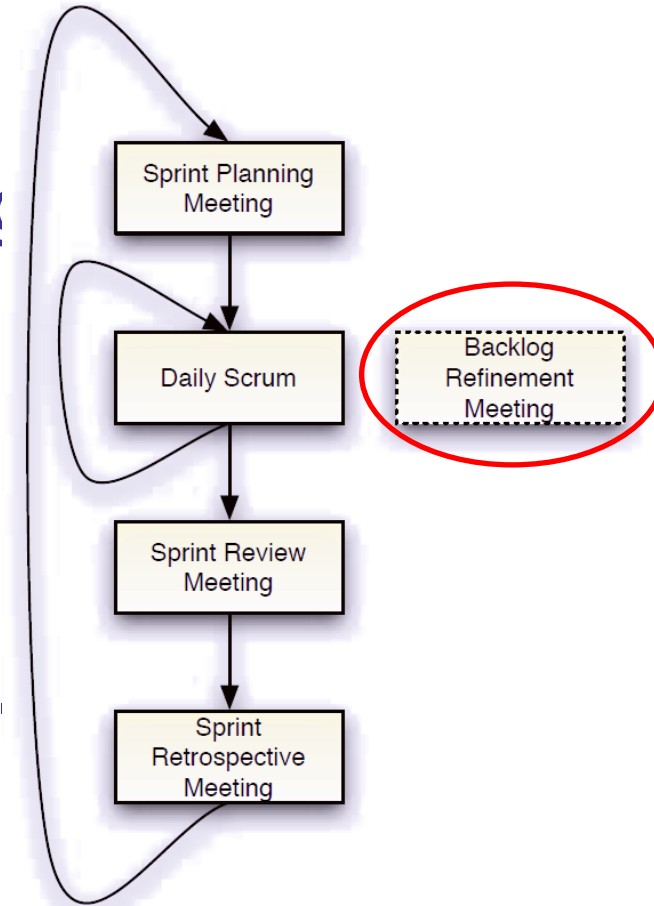
- At the end of every Sprint, the team meets to reflect on its own process. They inspect their own behavior and take action to adapt it for future Sprints.
- This meeting provides **an inspect-and-adapt** mechanism for the team's process. Common topics include:
 - "What went well?";
 - "What could be improved?";
 - "What did we learn?";
 - "What still puzzles us?";
 - "What actions will we take?"



Scrum Meetings (5)

Backlog Refinement Meeting

- It helps the Product Owner get the top of the Product Backlog ready for the next Sprint Planning Meeting.
- Backlog refinement includes estimation of effort, clarification of requirements, and decomposition of large Product Backlog Items (often called “epics”) into smaller ones (such as “user stories”).
- This meeting may also be called “Backlog Maintenance,” “Backlog Grooming,” “Backlog Estimation Meeting,” etc.

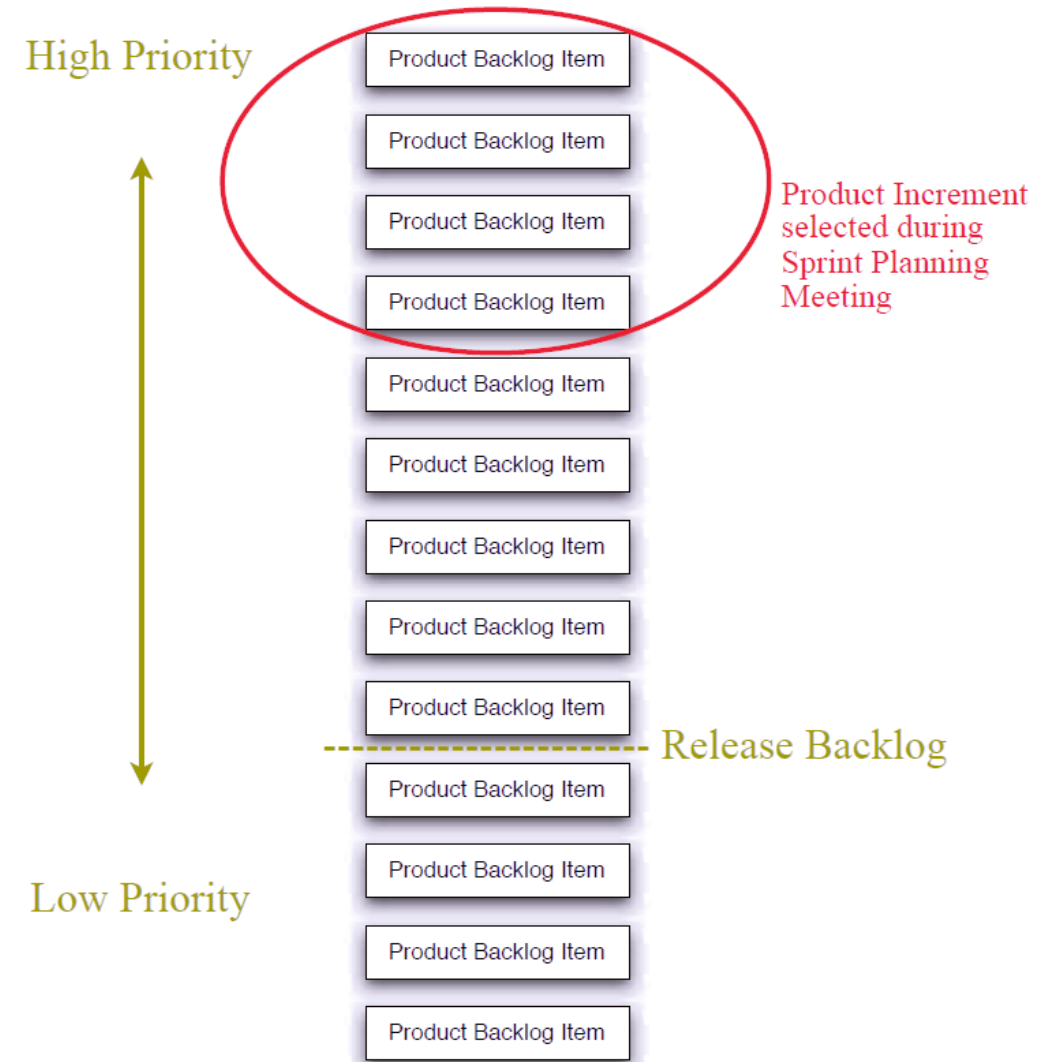


Scrum Artifacts (1)

• roles, meetings, and Artifacts

• Product Backlog

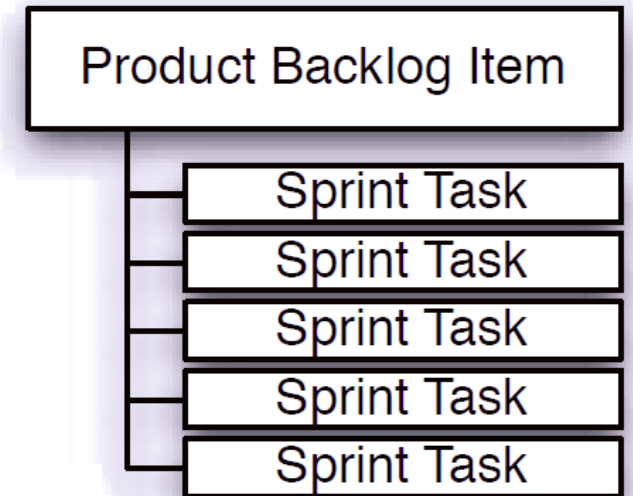
- A ranked list of desired functionality, constantly re-prioritized by Product Owner
- Visible to team members and all stakeholders. They can add items as well
- Maintained during Backlog Refinement Meeting.



Scrum Artifacts (2)

Product Backlog Item (PBI)

- Specifies the WHAT, not the HOW, of a customer centric feature
- Often written in "User Story" form
- Has acceptance criteria (and/or product-wide definition of "done") to prevent technical debt
- Usually requiring several tasks to achieve definition of "done"
- Effort is estimated by Team, ideally in relative unit, (e.g. story points)
- Business value is estimated by Product Owner Meeting.



Scrum Artifacts (3)

- **Sprint Backlog**

- Committed PBIs negotiated between Team and Product Owner during Sprint Planning Meeting
- Scope Commitment is fixed during Sprint Execution
- Initial tasks created by Team during Sprint Planning Meeting, and expected to change during Sprint Execution

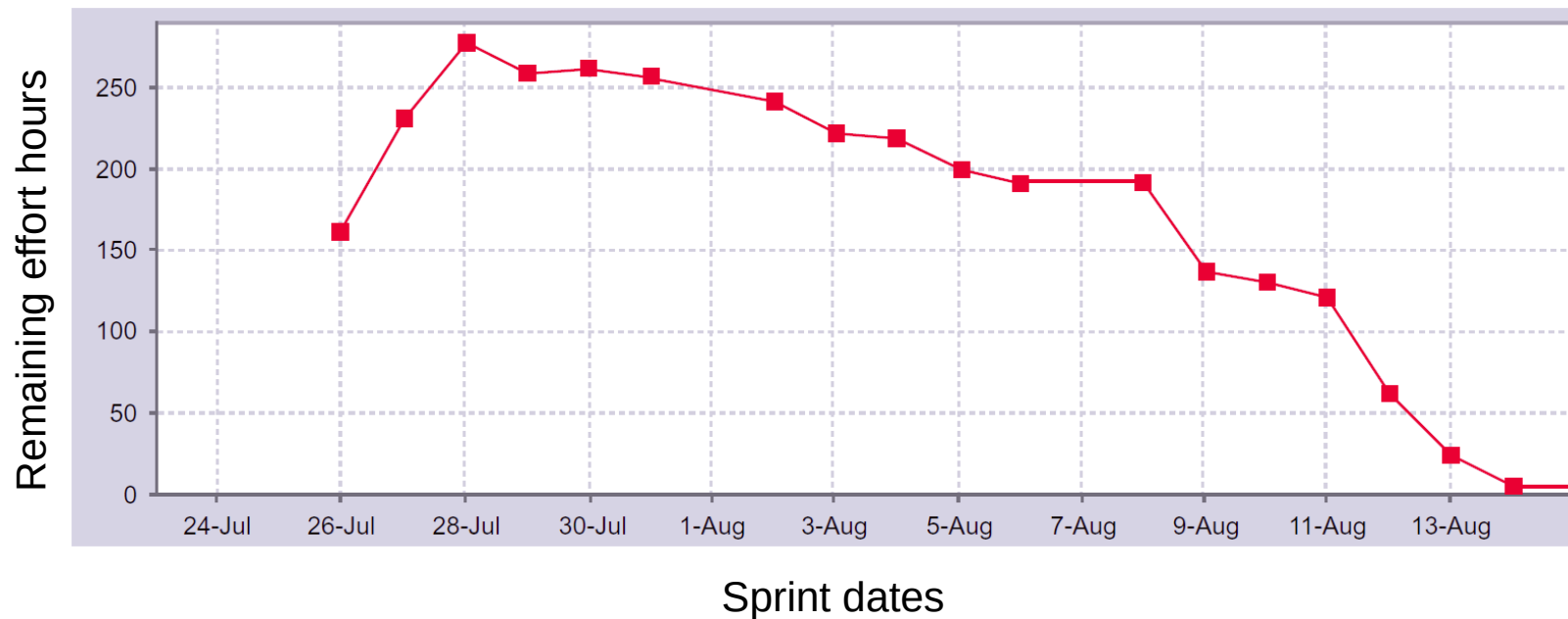
- **Sprint Task**

- Specifies “how” to achieve the PBIs’ “what”
- About one day or less of work
- Remaining effort re-estimated daily, typically in hours

Scrum Artifacts (4)

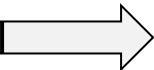
- **Sprint Burndown Chart**

- Total remaining team task hours within one Sprint
- Re-estimated daily, thus may go up before going down
- Intended to facilitate team self-organization, not a management report



Evaluating Agile Processes

- Agile processes are good when requirements are vague or changing
- Agile processes have had some successes with small-scale software development
 - However, medium- and large-scale software development are completely different
- The success of **pair programming** depends on both the programmer's expertise and the complexity of the tasks to be solved



Evaluating Agile Processes

- The key decider: the impact of agile processes on post-delivery maintenance
 - Refactoring is an essential component of agile processes
 - Refactoring continues during maintenance
 - Will refactoring increase the cost of post-delivery maintenance? (depends on the flexibility of the software design)

Selecting the Appropriate Development Methodology

- Since there are many methodologies, the first challenge faced by analysts is to select **which methodology to use**.
- But, no one methodology is always best (if it were, we'd simply use it everywhere!).
- Let us consider the following set of criteria that help in selecting an appropriate development methodology



Important Factors to Consider

- **Clarity of User Requirements**
- **Familiarity with Technology**
- **System Complexity**
- **System Reliability**
- **Short Time Schedules**
- **Schedule Visibility (Need to see progress on the time schedule)**

Selecting the Appropriate Development Methodology

Ability to develop systems	Structured methodology		RAD methodology				Agile methodology
	Waterfall	V-Model	Phased	System Prototyping	Throw-away Prototyping	Spiral	XP/Scrum
with Unclear User Requirements	Poor	Poor	V. Good	V. Good	Excellent	V. Good	Excellent
with Unfamiliar Technology	Poor	Poor	V. Good	Poor	Excellent	Excellent	Poor
that are Complex	V. Good	V. Good	V. Good	Poor	Excellent	Excellent	Poor
that are Reliable	Good	Excellent	Good	Poor	V. Good	Excellent	Good
with Short Time Schedules	Poor	Poor	Excellent	Excellent	Poor	Poor	Excellent
with Schedule Visibility	Poor	Poor	Excellent	Excellent	Good	Good	V. Good

Questions?

